

UNIT-II SUPERVISED LEARNING (AY-2026-27)

**Regression, Cost function, Gradient
Descent, Bias-Variance Tradeoff,
Overfitting ,
Underfitting, Regularization**

Regression

- Regression is a supervised learning technique used to predict continuous numerical values by learning relationships between **input variables (features)** and an **output variable (target)**.
- It helps understand how changes in one or more **factors influence a measurable outcome** and is widely used in **forecasting, risk analysis, decision-making and trend estimation**.

How and where it works?

- ✓ Works with real valued output variables
- ✓ Helps to identify strengths and the type of relationships
- ✓ Supports both simple and complex predictive models.
- ✓ Used for tasks like price prediction, trend forecasting and risk scoring.

Types of Regression

❑ Regression can be classified into different types based on the **number of predictor variables** and the **nature of the relationship between variables**:

1. **Simple Linear Regression**
2. **Multiple Linear Regression**
3. **Polynomial Regression**
4. **Ridge and Lasso Regression**
5. **Support Vector Regression**
6. **Decision Tree Regression**
7. **Random forest Regression**

1. Simple Linear Regression

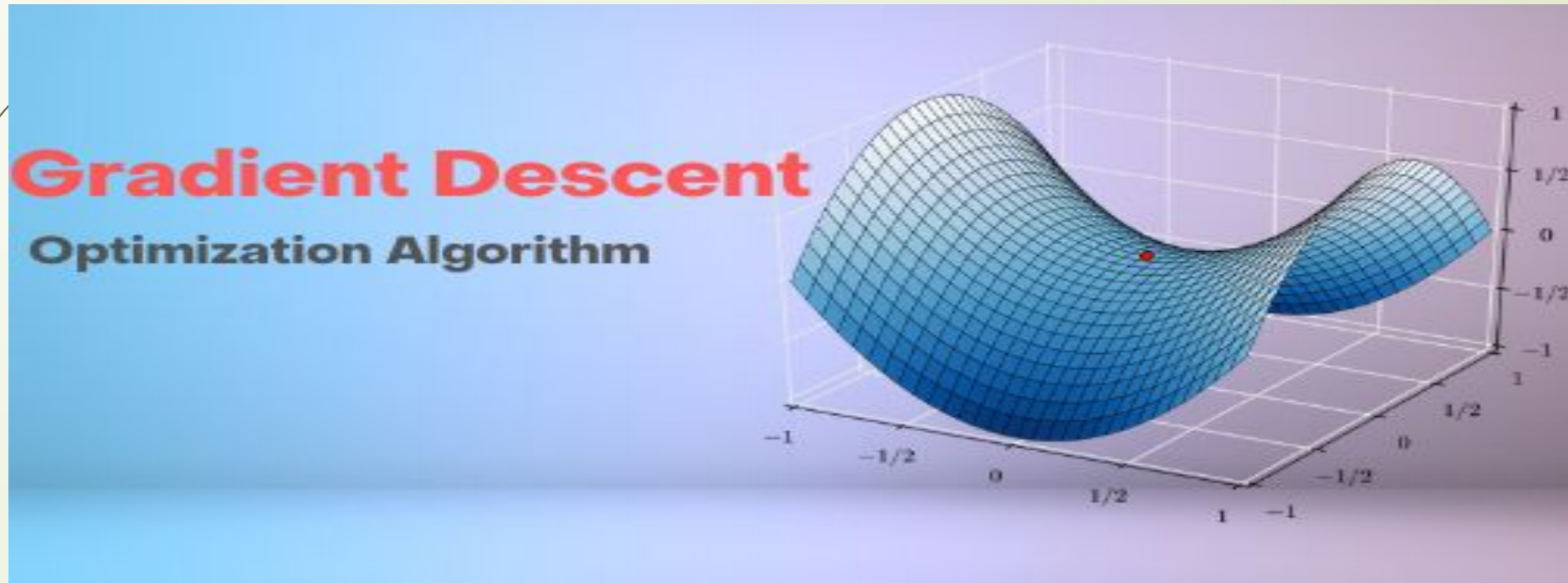
- models the relationship between one independent variable and a continuous dependent variable by fitting a straight line that minimizes the sum of squared errors. It assumes a constant rate of change, meaning the output varies proportionally with the input.
- **Application:** Estimating house price from only its size
- **Advantage:** Highly interpretable due to its simple mathematical structure
- **Disadvantage:** Cannot capture curved or complex data patterns



Gradient Descent for Convex Functions:

- **Gradient Descent** is an optimization algorithm used to **minimize** a function by updating parameters in the **opposite direction of the gradient** of the loss function.
- 

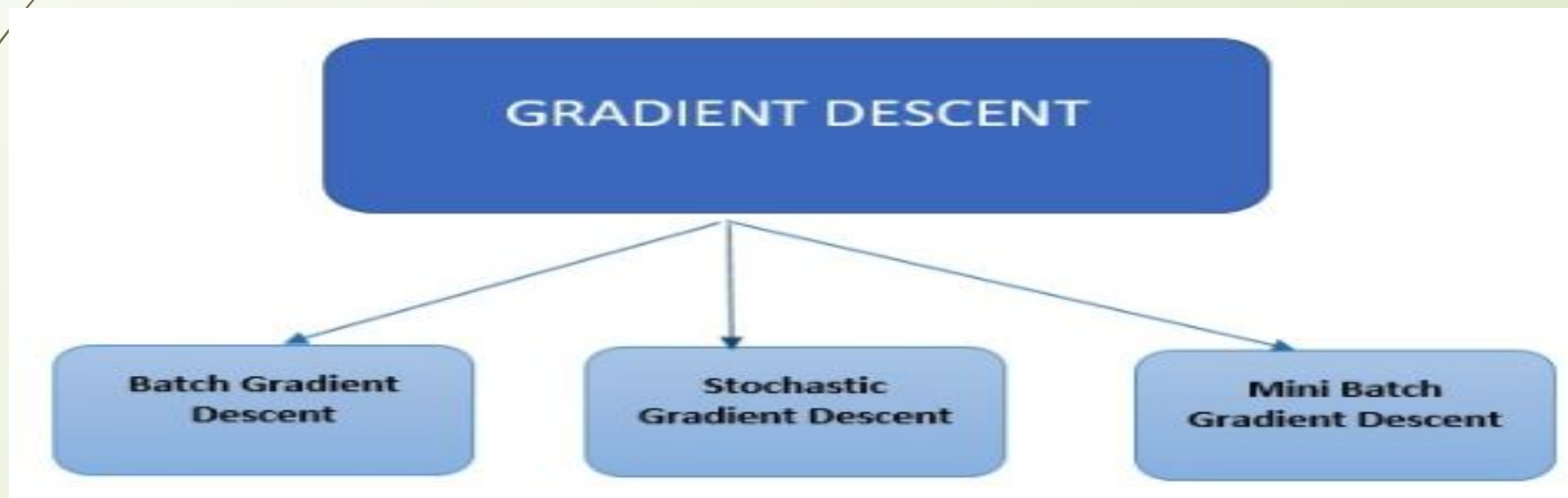
□ Gradient Descent is an **optimization algorithm** that is used **to minimize a function** by slowly moving in the direction of steepest descent, which is defined by the **negative of the gradient**.



Types of Gradient Descents

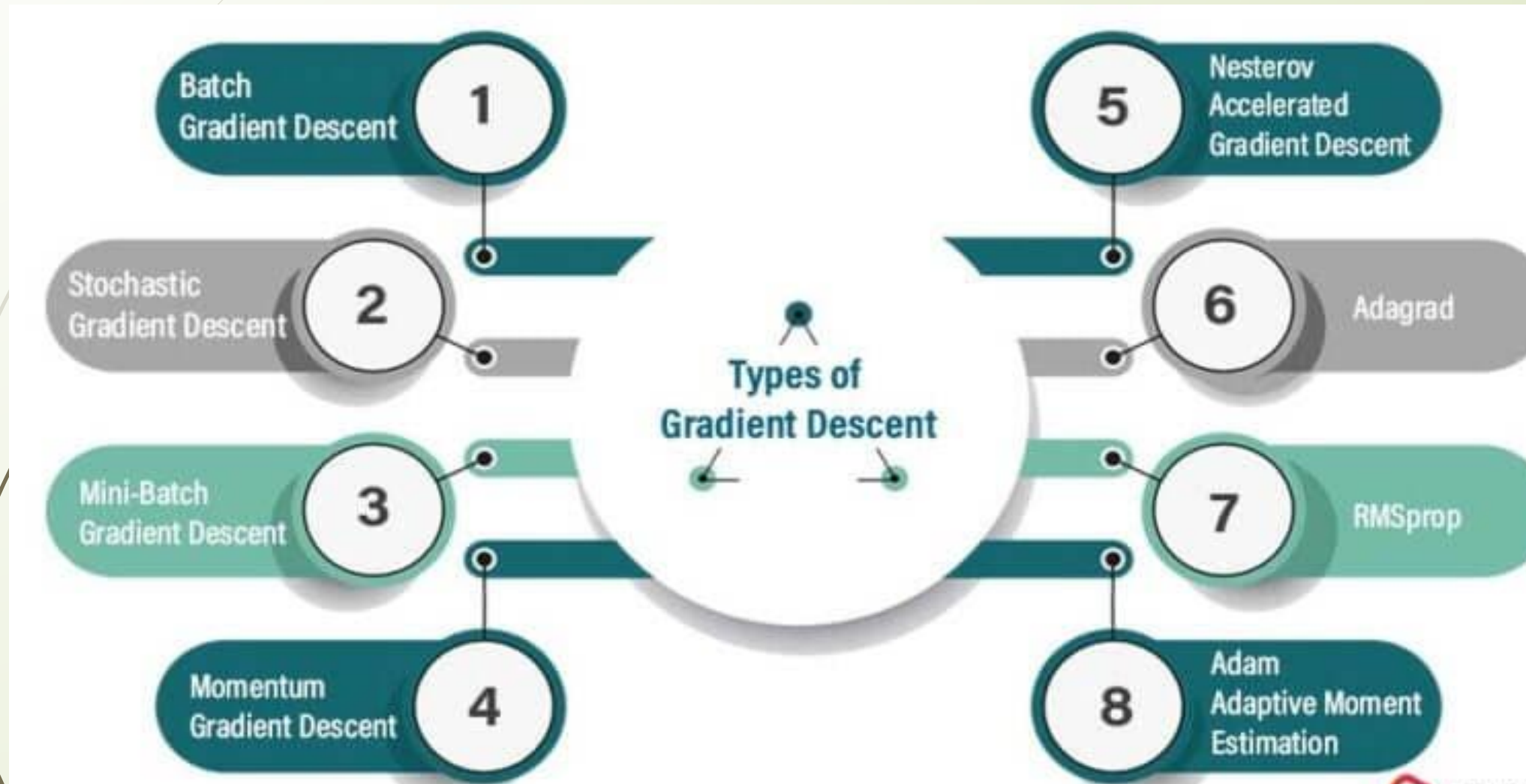
3 types of gradient descent:

- ❑ 1. Stochastic Gradient Descent
- ❑ 2. Batch Gradient Descent
- ❑ 3. Mini-Batch Gradient Descent



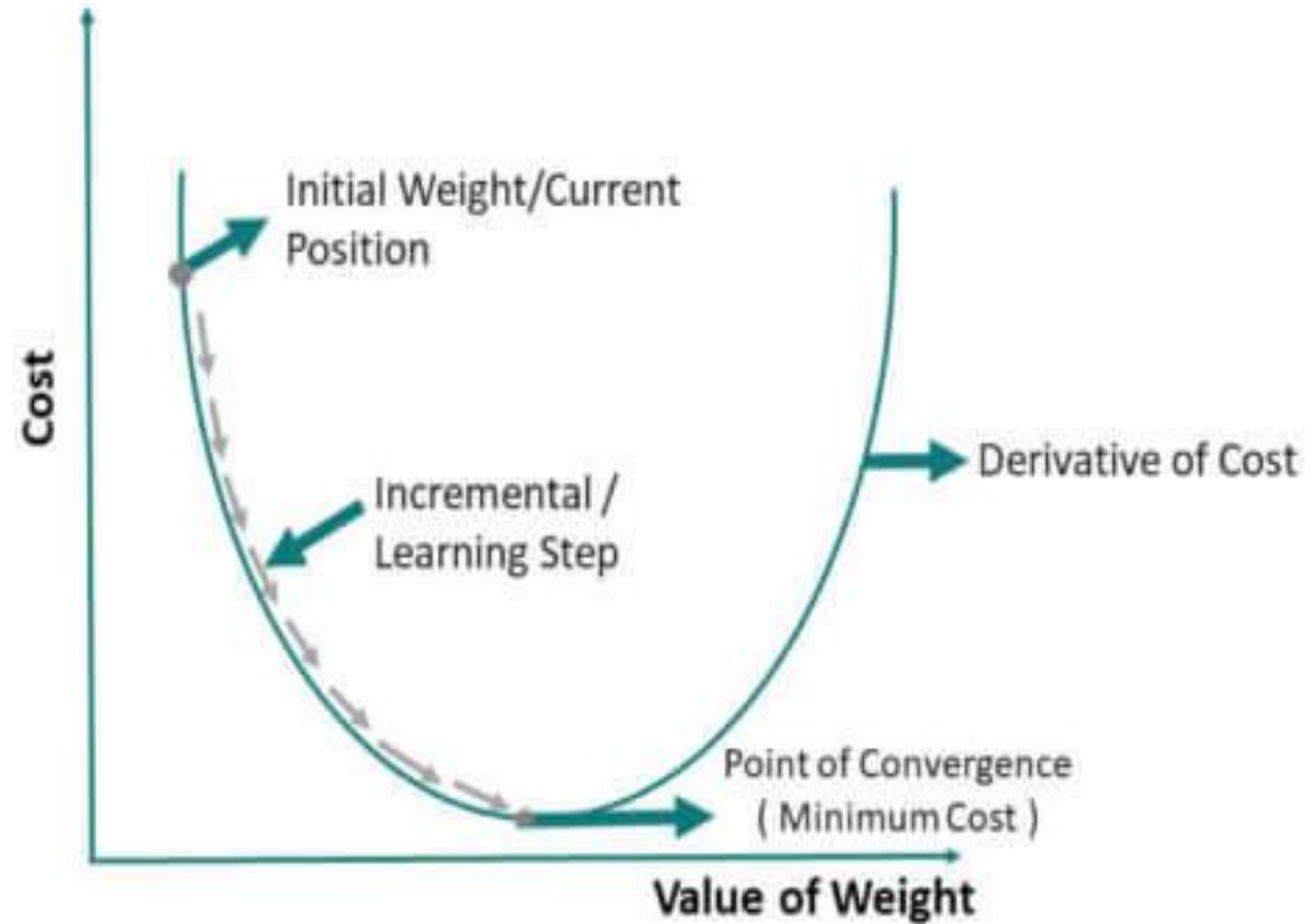
- 
- ❑ **Stochastic Gradient Descent** — in extreme cases, gradient descent picks one instance of training data at every step and update based on that one instance of data point
 - ❑ **Batch Gradient Descent** — in batch gradient descent, the algorithm uses whole data-set to update the values of coefficients.
 - ❑ **Mini-Batch Gradient Descent** — in mini-batch gradient descent, algorithm picks a mini batch from the whole data-set at every step and update the values of coefficients. This method has the advantages of both stochastic and batch gradient descent.

Different Types of Gradient Descent



Variant	Advantages	Disadvantages
Batch gradient descent	Guaranteed convergence to global optimum	Computationally expensive for large datasets, slow convergence
Stochastic gradient descent	Faster convergence, more efficient for large datasets	High variance, may not converge to global optimum
Mini-batch gradient descent	Balanced convergence speed and computational cost, efficient for large datasets	Choice of mini-batch size can be a challenge
Momentum gradient descent	Faster convergence, less likely to get stuck in local minima	May overshoot and oscillate around the optimum
Adagrad	Adaptive learning rate, efficient for sparse data	Can stop learning too early
RMSProp	Adaptive learning rate, efficient for non-stationary problems	Can stop learning too early, requires tuning of hyperparameters
Adam	Adaptive learning rate, efficient for large datasets and noisy data	Can converge to suboptimal solutions, requires tuning of hyperparameters

Gradient Descent of Machine Learning

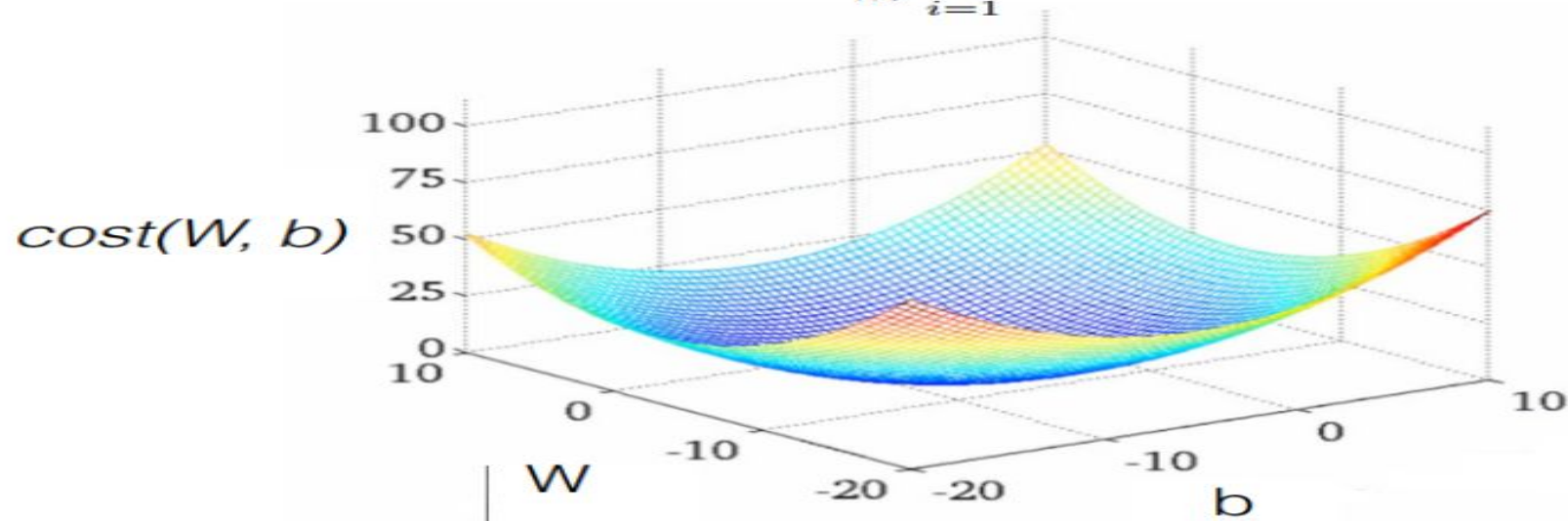


What is Convex Function?

- A **convex function** is a function where the line segment between any two points on the graph lies **above** or **on the graph** itself.

Convex function

$$cost(W, b) = \frac{1}{m} \sum_{i=1}^m (H(x^{(i)}) - y^{(i)})^2$$





Properties of Convex Functions

Property

Explanation

Single global minimum

Only one lowest point; no local minima

Second derivative ≥ 0

$f''(x) \geq 0$ for all x

Gradient always increasing

Slope gets steeper as x increases

Easy to optimize

Gradient descent always converges to the global minimum



Why Are Convex Functions Important in ML?

- Most machine learning algorithms (like **linear regression, logistic regression**) use convex cost functions.
- Convexity ensures **easy optimization** with algorithms like **gradient descent**.
- It don't get stuck in **local minima**.



Real-World Applications of Convex Cost Functions:

Application	Convex Function Used	Algorithm
House price prediction	MSE	Linear Regression
Spam classification	Log Loss (cross-entropy)	Logistic Regression
Face recognition	Hinge Loss	SVM
Resource allocation in cloud	Quadratic Cost Functions	Optimization Systems

Real Time Example:

- Let's say we want to predict the price of a house based on its size (in square feet).
- We can use Linear Regression, where the cost function used is a convex function.
- 1. Predict house price using:
- 2. Model (Hypothesis Function):

$$\hat{y} = w_0 + w_1x$$

- Input: x = size of the house
- Output: y = price of the house

Where:

- $\hat{y}$: predicted price
- w_0 : intercept
- w_1 : slope (how much price changes per sq.ft)



This is a **convex function**. Its plot looks like a **bowl-shaped curve**, and it has **only one global minimum**.

Since, the MSE is **convex**, algorithms like **gradient descent** can find the best w_0, w_1 easily.

We use Mean Squared Error (MSE) as the loss function:

$$J(w_0, w_1) = \frac{1}{n} \sum_{i=1}^n (\hat{y}_i - y_i)^2$$

Logistic Regression:

▣ **Purpose:** Used for binary classification (e.g., spam or not spam).

▣ **Concept:**

1. Logistic regression uses the **logistic (sigmoid) function** to predict probabilities.

2. It models the probability that an instance belongs to a particular class.

▣ **Formula:**

$$P(Y = 1) = \frac{1}{1 + e^{-(\beta_0 + \beta_1 X_1 + \dots + \beta_n X_n)}}$$

Decision Tree

- ❑ **Main key definition:**

- ❑ Output is between 0 and 1 (probability).
- ❑ If probability $> 0.5 \rightarrow$ class 1, else class 0.
- ❑ Assumes linear relationship between input features and log-odds.

- ❑ **Examples:**

- ❑ Email spam detection
- ❑ Disease prediction (e.g., diabetes: yes/no)
- ❑ Loan approval



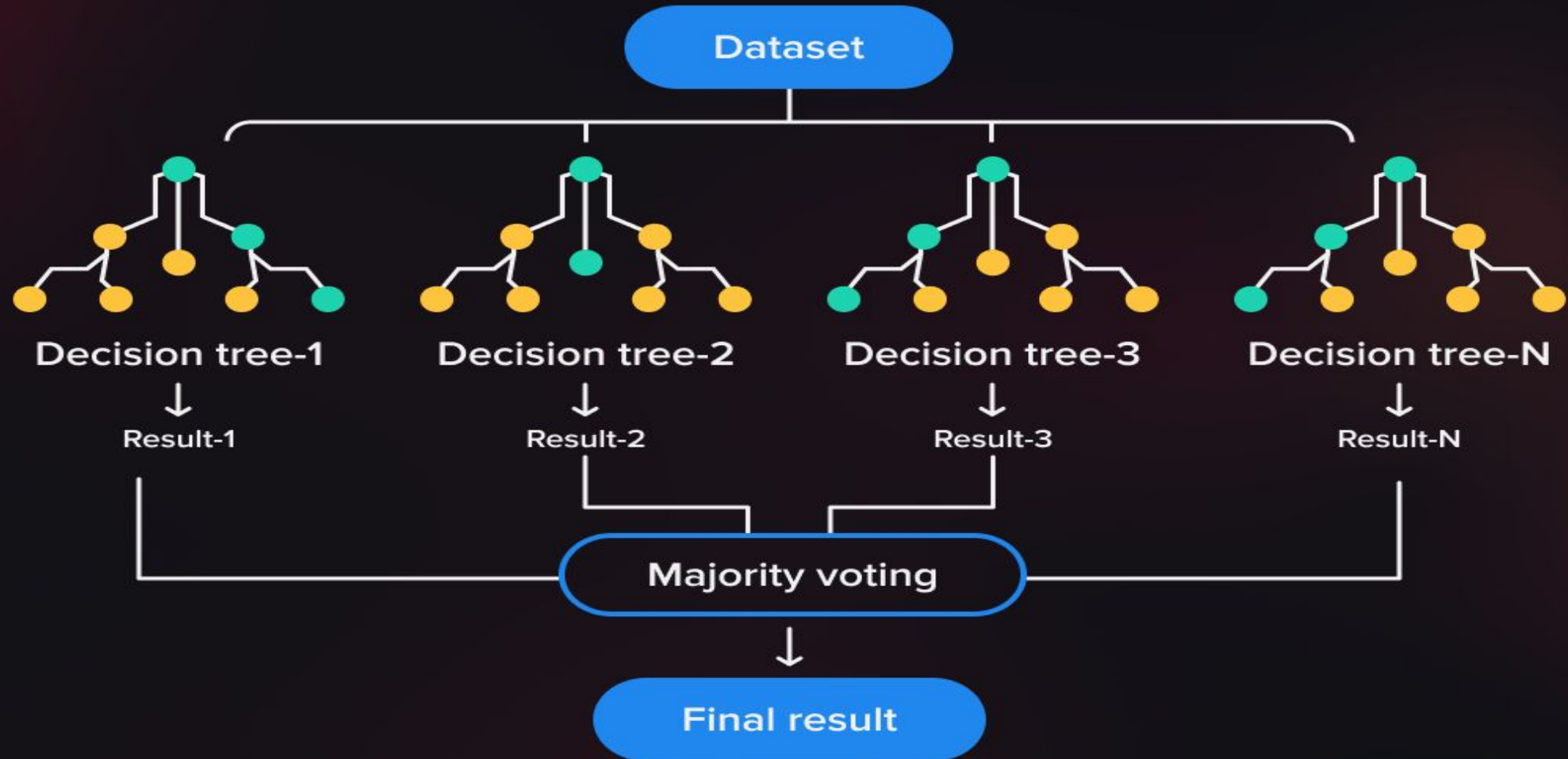
Decision Tree

- ❑ **Purpose:** Used for both classification and regression.
- ❑ **Concept:**
 - ❑ A tree-like structure where each internal node tests a feature.
 - ❑ Branches represent outcomes of the test.
 - ❑ Leaf nodes represent class labels or predicted values.



A **decision tree** is a type of machine learning model that is used when the relationship between a set of predictor variables and a response variable is non-linear.

Decision Tree



□ How it works:

- Uses measures like **Gini Impurity** or **Entropy** to split nodes.
- Greedy algorithm (splits that give the best immediate gain).

□

Is age > 30?

□

Yes: Is income > 50k?

□

Yes → Class A

□

No → Class B

□

No → Class C



□ Advantages:

- Easy to understand and visualize.
- Handles both numerical and categorical data.

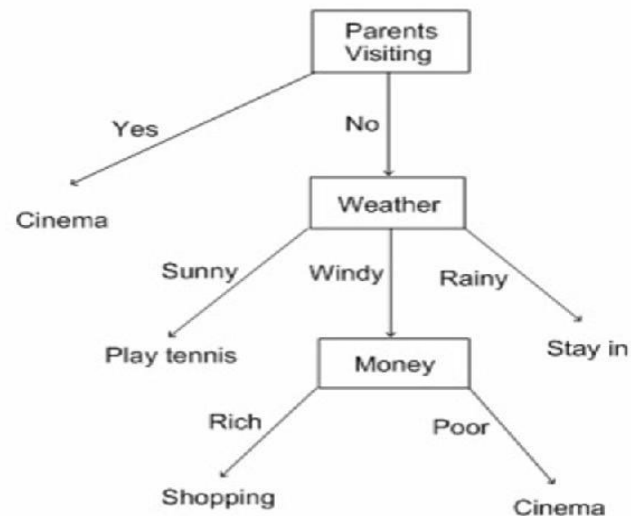
□ Limitations:

- Prone to **over fitting**.

Random Forest

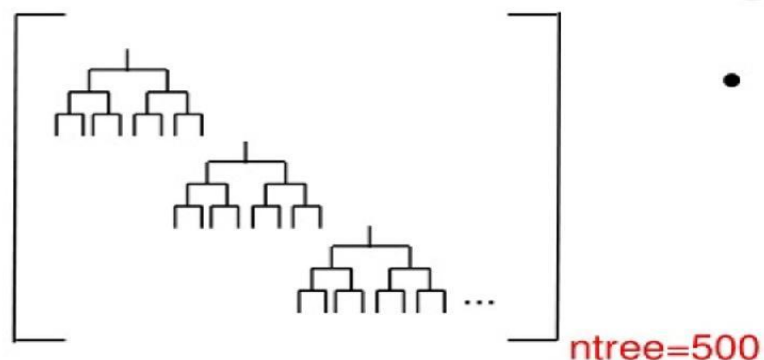
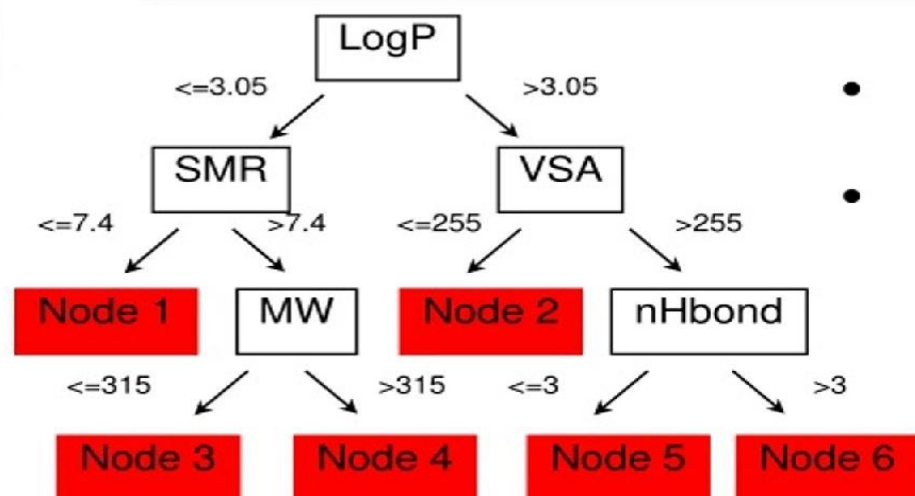
- ❑ **Purpose:** An ensemble method for classification and regression.
- ❑ **Concept:**
 - ❑ Combines **many decision trees** (a "forest").
 - ❑ Each tree is trained on a random subset of data and features (bootstrap).
 - ❑ Final output:
 1. Classification → **majority voting**
 2. Regression → **average prediction**

Random Forest (Classification)



- Coupled Ensemble of Decision Trees
 - Each tree is trained:
 - from a bootstrap sample of the data
 - *in situ* out-of-bag cross-validation
 - without pruning back;
 - for classification typically **nodesize=1**
 - from subset of descriptors at each split;
- for classification typically **$m_{try} = \text{SQRT}(\text{no. of descriptors})$**
- Advantages:
 - improved accuracy
 - method for descriptor selection
 - no overfitting
 - easy to train
 - human interpretable
 - not a black box

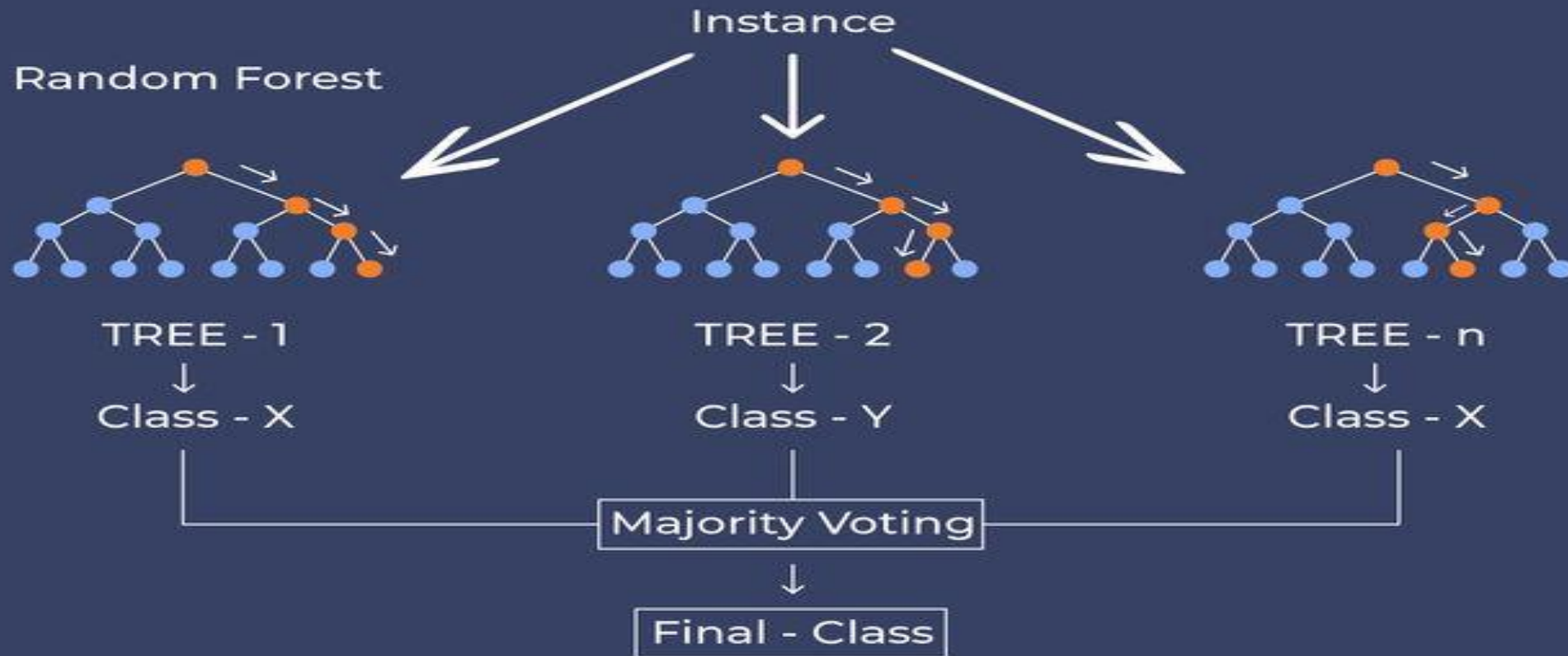
Random Forest (Regression)



- Coupled Ensemble of Regression Trees
- Each tree is trained:
 - from a bootstrap sample of the data *in situ* out-of-bag cross-validation
 - without pruning back;
 - for regression typically **nodesize=5**
 - from subset of descriptors at each split;
 - for regression typically **mtry=(no. of descriptors)/3**
- Advantages:
 - improved accuracy
 - method for descriptor selection
 - no overfitting
 - easy to train
 - human interpretable
 - not a black box

RANDOM FOREST

CLASSIFICATION



❑ **Reduces over fitting of individual trees.**

❑ **Advantages:**

❑ More accurate and stable than a single decision tree.

❑ **Key Concepts:**

❑ **Bagging** (Bootstrap Aggregating)

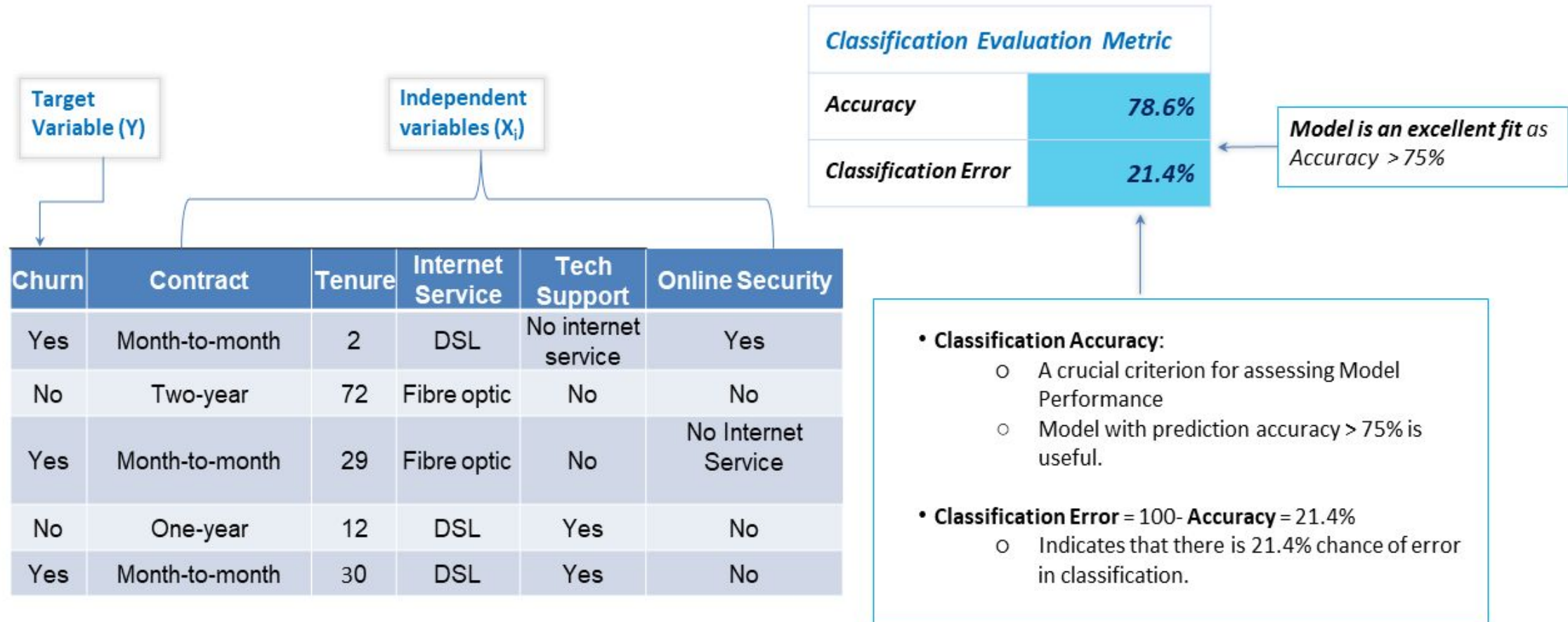
❑ Randomness in feature selection and data sample improves generalization.

❑ **Use Cases:**

1. Fraud detection
2. Medical diagnosis
3. Stock market prediction

Example: Random Forest Classification

Let's conduct the Random Forest Classification analysis on **independent variables**: *Contract, Tenure, Internet Service, Tech Support, Online Security* and **target variable**: *Churn* as shown below:



Difference Between Random Forest and Decision Tree

Property	Random Forest	Decision Tree
Nature	Ensemble of multiple decision trees	Single Decision Tree
Interpretability	Less interpretable due to ensemble nature.	Highly interpretable.
Overfitting	Due to ensemble averaging it is less prone to overfitting.	More prone to overfitting specially in case of deep trees.
Training Time	Since multiple trees are constructed, training time becomes more, and training speed becomes less.	A single tree needs to be built and trained, hence faster in comparison.
Stability to change	Since overall average is taken due to ensemble, it is more stable to change.	It becomes quite sensitive to variation in data.
Predictive Time	Multiple predictions, hence longer prediction time and slower prediction speed.	Faster prediction as compared to random forest, since a single prediction is made.
Performance	Generally performs well on large datasets.	It can perform well on small and large dataset as well.
Handling Outliers	Due to ensemble averaging more robust to outliers.	It is more susceptible to outliers.
Feature Importance	Do not provide feature score directly rather uses ensemble to decide feature score.	Provide feature score directly which are less reliable.

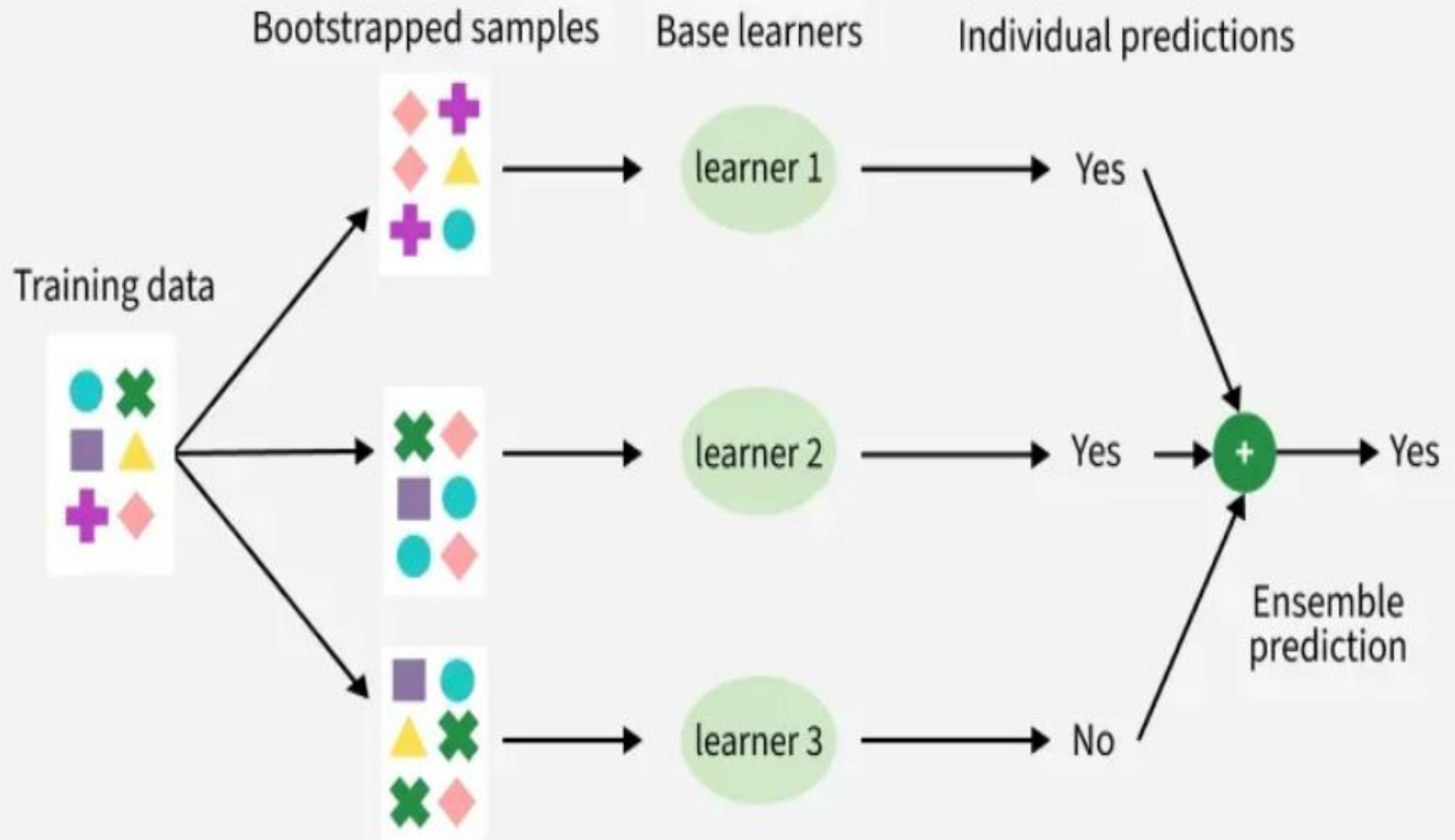
Ensemble learning

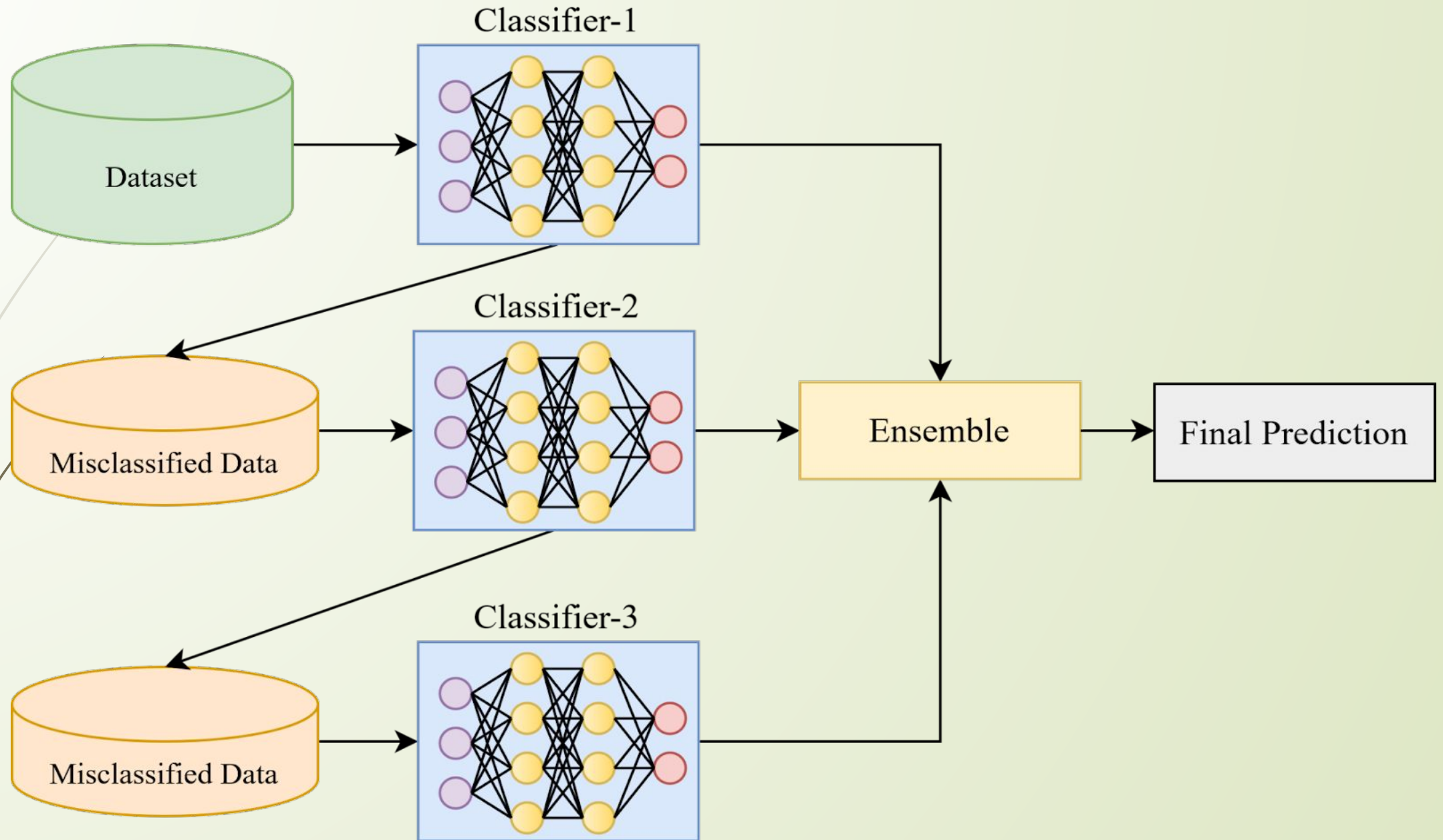
- **Ensemble learning** is a technique in machine learning where **multiple models (often called "weak learners")** are trained and combined to solve the same problem.
- The idea is that **a group of models working together can outperform a single strong model.**

When to Use Ensemble Learning?

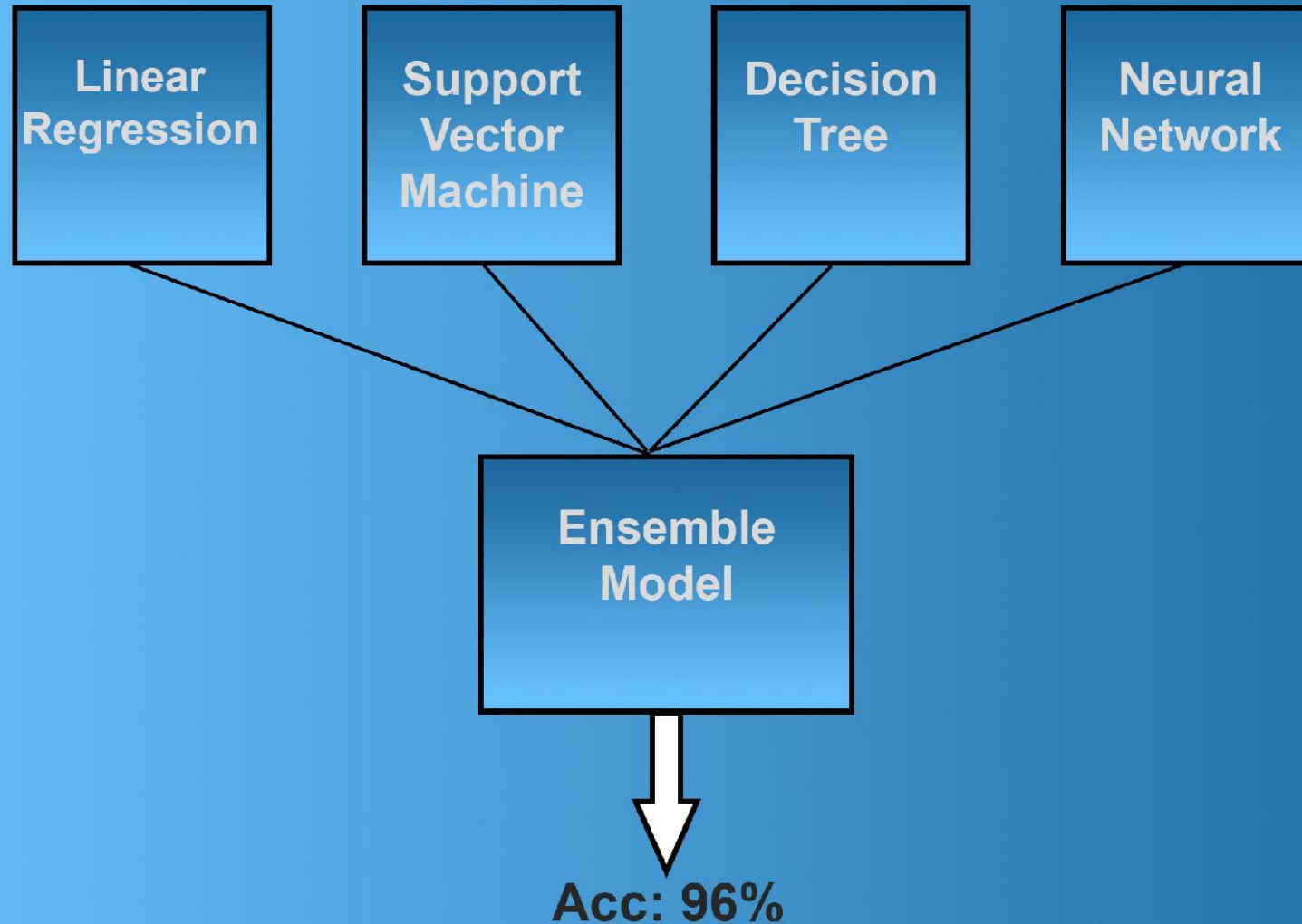
- ❑ You have **high variance or high bias** in your model.
- ❑ Your base models are **diverse and complementary**.
- ❑ You want to **increase model performance** in competitions (e.g., Kaggle).
- ❑ **Real-Life Example**
 - ❑ Imagine trying to guess a movie's rating:
 - ❑ One friend uses past ratings
 - ❑ Another reads online reviews
 - ❑ A third watches the trailer
 - Each has weaknesses alone, but combining their opinions gives a better estimate. That's ensemble learning!

Ensemble learning

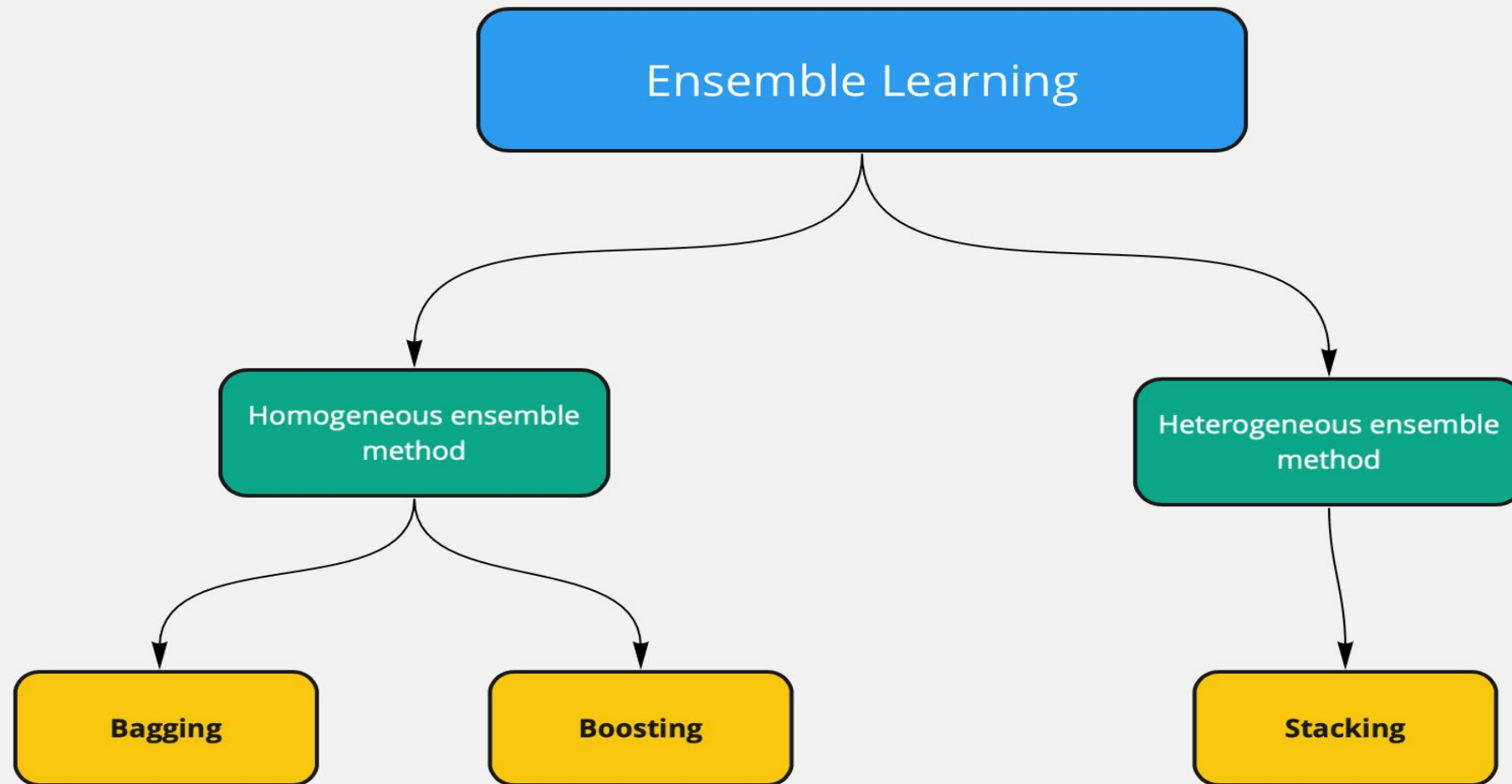




Accuracy: Highest



Types of Ensemble learning

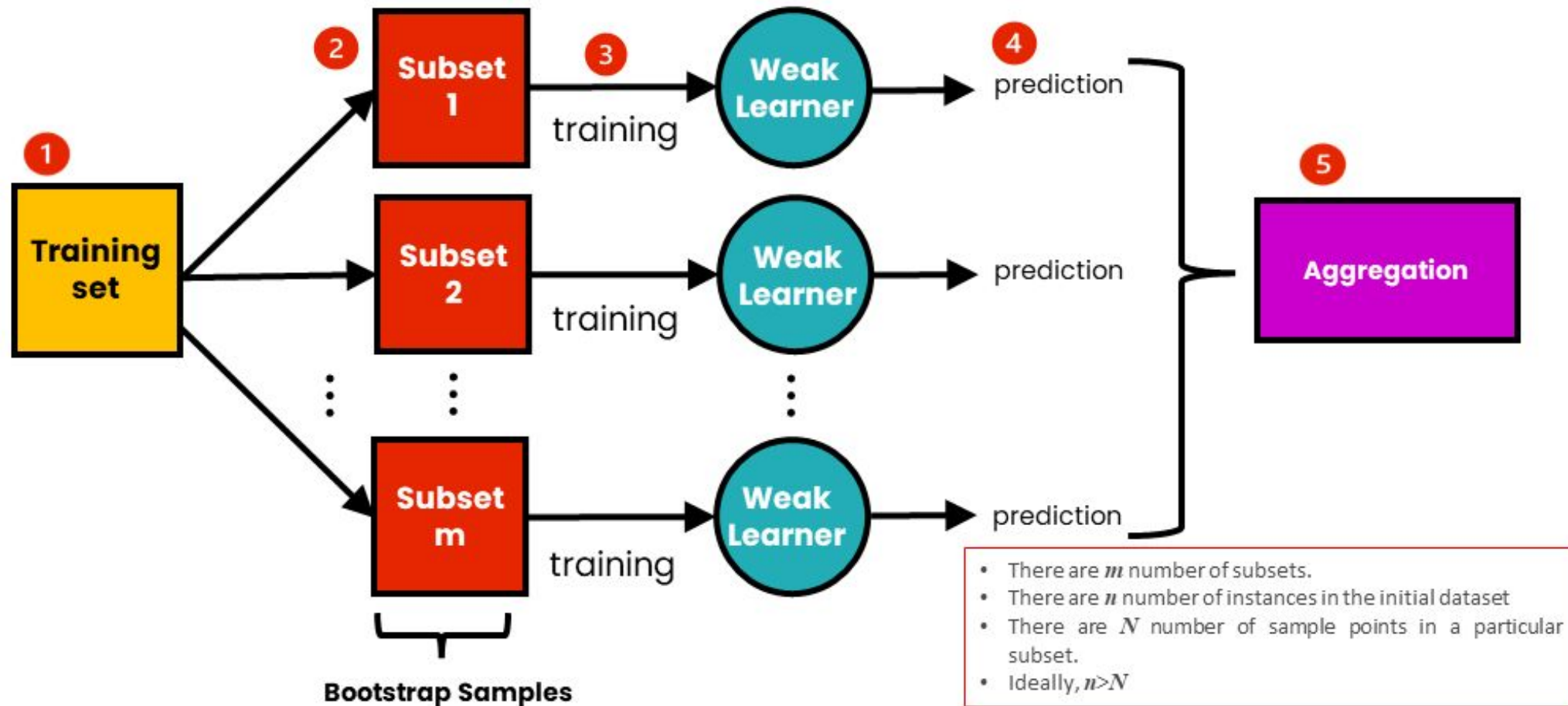


❑ **Bagging (Bootstrap Aggregating):**

- Models are trained independently on different random subsets of the training data.
- ❑ Their results are then combined—usually by averaging (for regression) or voting (for classification).
- ❑ This helps reduce variance and prevents over fitting.
- ❑ **Example Algorithms:**
- ❑ **Random Forest** (uses decision trees + bagging)

How Bagging Learning model works:

The Process of Bagging (Bootstrap Aggregation)

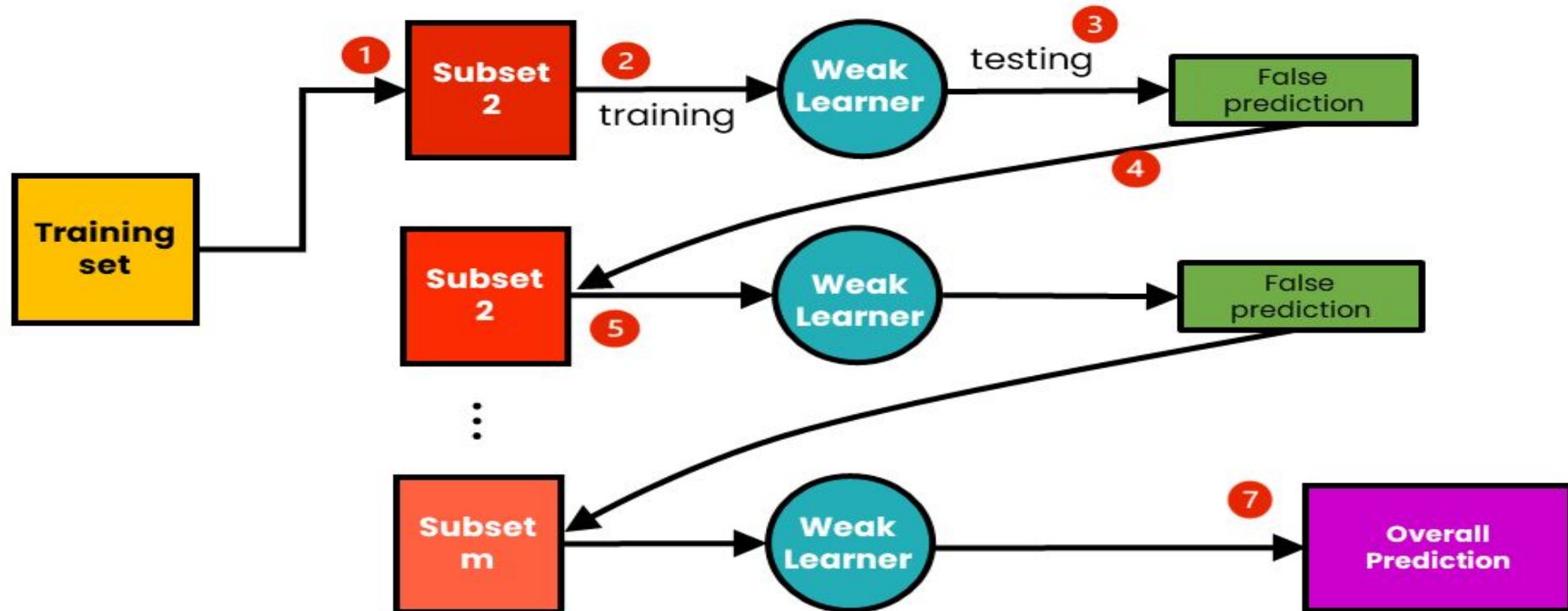


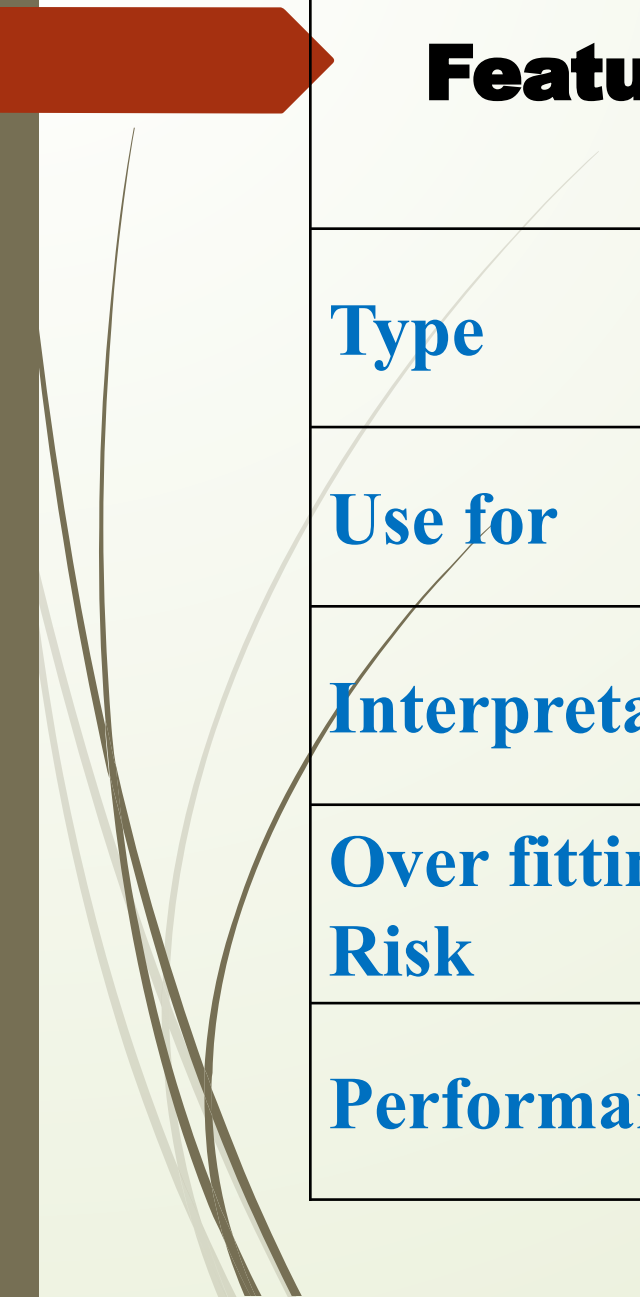
Boosting:

Models are trained one after another. Each new model focuses on fixing the errors made by the **previous ones**.

The final prediction is a weighted combination of all models, which helps **reduce bias and improve accuracy**.

The Process of Boosting





Feature	Logistic Regression	Decision Tree	Random Forest
Type	Linear Model	Non-linear Tree	Ensemble of Trees
Use for	Classification	Both	Both
Interpretability	High	Moderate	Low (Black Box)
Over fitting Risk	Low	High	Low
Performance	Moderate	Fast but unstable	High accuracy

Definition

In data science and statistics, **regression** is a **predictive modeling technique** used to estimate the relationship between a dependent variable (the outcome you want to predict) and one or more independent variables (the factors influencing the outcome)

Linear Regression

Linear Regression is a fundamental supervised learning algorithm used to model the relationship between a **dependent variable and one or more independent variables**. It predicts continuous values by fitting a straight line that best represents the data.

1. It assumes that there is a linear relationship between the input and output
2. Uses a best-fit line to make predictions
3. Commonly used in forecasting, trend analysis, and predictive modelling



For example, suppose we want to predict a student's exam score based on the number of hours studied. As study hours increase, exam scores generally increase as well.

Here:

- **Independent variable (input):** Hours studied because it's the factor we control or observe.
- **Dependent variable (output):** Exam score because it depends on how many hours were studied.

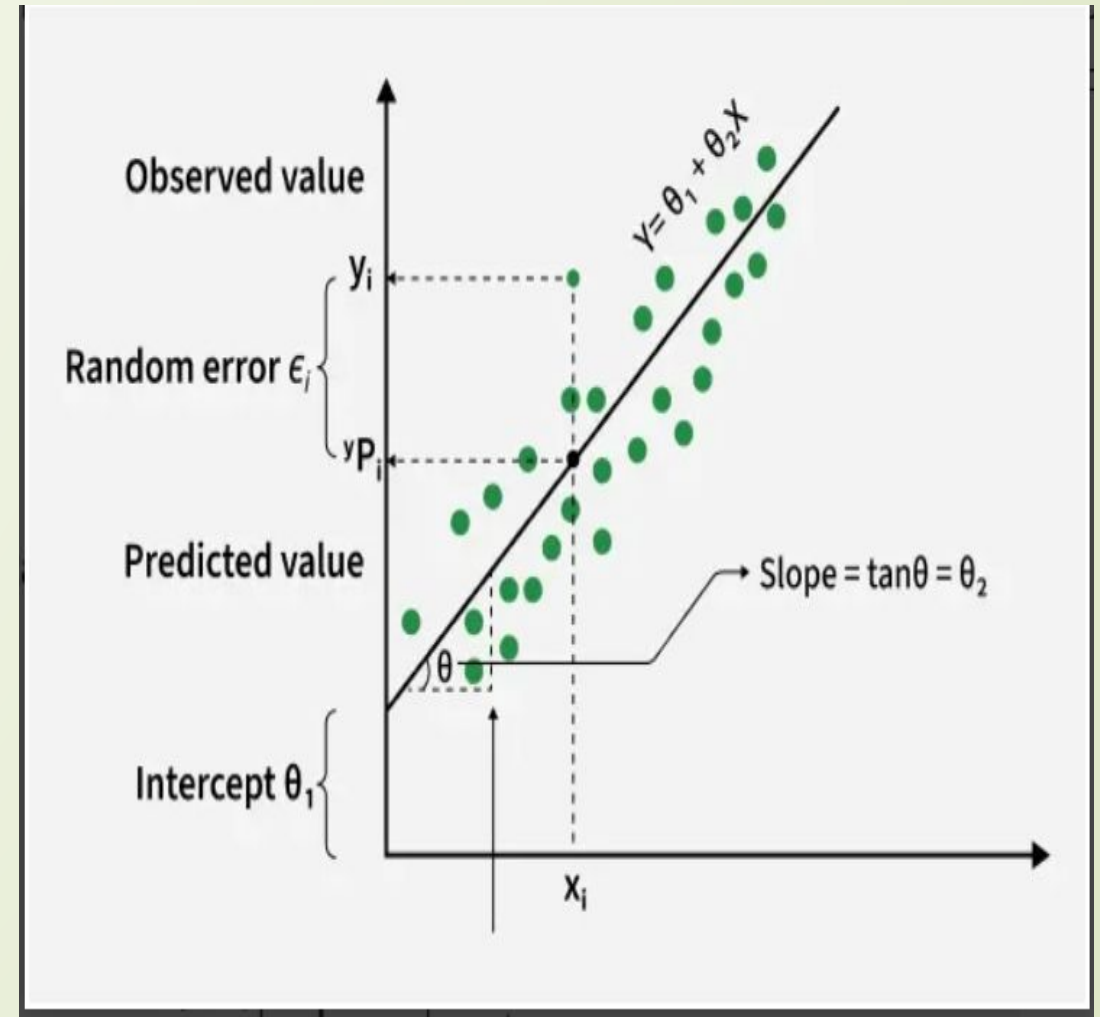
Linear regression uses the independent variable to predict the dependent variable.

Best Fit Line

- In linear regression, the best-fit line is the straight line that best represents the relationship between the independent variable (input) and the dependent variable (output).
- The goal **is to minimize the difference between the actual data points and the predicted values generated by the model.**

1. Goal of the Best-Fit Line

- The goal of linear regression is **to find a straight line that minimizes the error (the difference) between the observed data points and the predicted values**. This line helps us **predict the dependent variable for new, unseen data**.





Here Y is called a dependent or target variable and X is called an independent variable also known as the predictor of Y .

1. θ_1 represents the intercept, which is the value of Y when $X = 0$
2. θ_2 represents the slope, which shows how much Y changes for a unit change in X
3. There are many types of functions or modules that can be used for regression. A linear function is the simplest type of function. Here, X may be a single feature or multiple features representing the problem.

2. Equation of the Best-Fit Line

For simple linear regression (with one independent variable), the best-fit line is represented by the equation:

$$y=mx+c$$

Where:

- **y** is the predicted value (dependent variable)
- **x** is the input (independent variable)
- **m** is the slope of the line (how much y changes when x changes)
- **b** is the intercept (the value of y when $x = 0$)
- The best-fit line will be the one that optimizes the values of m (slope) and b (intercept) so that the predicted y values are as close as possible to the actual data points.

3. Minimizing the Error using the Least Squares Method

- To determine the best-fit line, linear regression uses the Least Squares Method, which minimizes the difference between actual and predicted values. These differences are called residuals.
- The formula for residuals is:

$$\text{Residual} = y_i - \hat{y}_i$$

Where:

- y_i is the actual observed value
- $\hat{y}_i$ is the predicted value from the line for that x_i

The least squares method minimizes the sum of the squared residuals:

$$\Sigma(y_i - \hat{y}_i)^2$$

This method ensures that the line best represents the data where the sum of the squared differences between the predicted values and actual values is as small as possible.

4. Interpretation of the Best-Fit Line

1. **Slope (m):** The slope indicates how much the dependent variable changes for every one-unit increase in the independent variable. For example, if the slope is 5, then y increases by 5 units for every 1-unit increase in x .
2. **Intercept (b):** The intercept represents the predicted value of y when $x = 0$. It's the point where the line crosses the y -axis.
3. In linear regression some hypothesis are made to ensure reliability of the model's results.
4. **Limitations:**

- *Assumes Linearity:* The method assumes the relationship between the variables is linear. If the relationship is non-linear, linear regression might not work well.
- *Sensitivity to Outliers:* Outliers can significantly affect the slope and intercept, skewing the best-fit line.

Hypothesis Function

- In linear regression, the hypothesis function is the equation **used to make predictions about the dependent variable based on the independent variables**. It represents the relationship between the input features and the target output.
- For a simple case with **one independent** variable, the hypothesis function is:

$$h(x) = \beta_0 + \beta_1 x$$

For one independent variable

For a simple case with one independent variable, the hypothesis function is:

$$h(x) = \beta_0 + \beta_1 x$$

Where:

- $h(x)$ or $(\hat{y})$ is the predicted value of the dependent variable (y).
- x is the independent variable.
- β_0 is the intercept, representing the value of y when x is 0.
- β_1 is the slope, indicating how much y changes for each unit change in x .

For more than one independent variable

For multiple linear regression (with more than one independent variable), the hypothesis function expands to:

$$h(x_1, x_2, \dots, x_k) = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \dots + \beta_k x_k$$

Where:

- $x_1, x_2, \dots, x_k$ are the independent variables.
- β_0 is the intercept.
- $\beta_1, \beta_2, \dots, \beta_k$ are the coefficients, representing the influence of each respective independent variable on the predicted output.

Cost Function

$$\text{Cost function}(J) = \frac{1}{n} \sum_n^i (\hat{y}_i - y_i)^2$$

- ❑ In Linear Regression, the cost function measures how far the predicted values $\hat{Y}$ are from the actual values (Y).
- ❑ It helps identify and reduce errors to find the best-fit line.
- ❑ The most common cost function used is **Mean Squared Error (MSE)**, which calculates the average of squared differences **between actual and predicted values.**

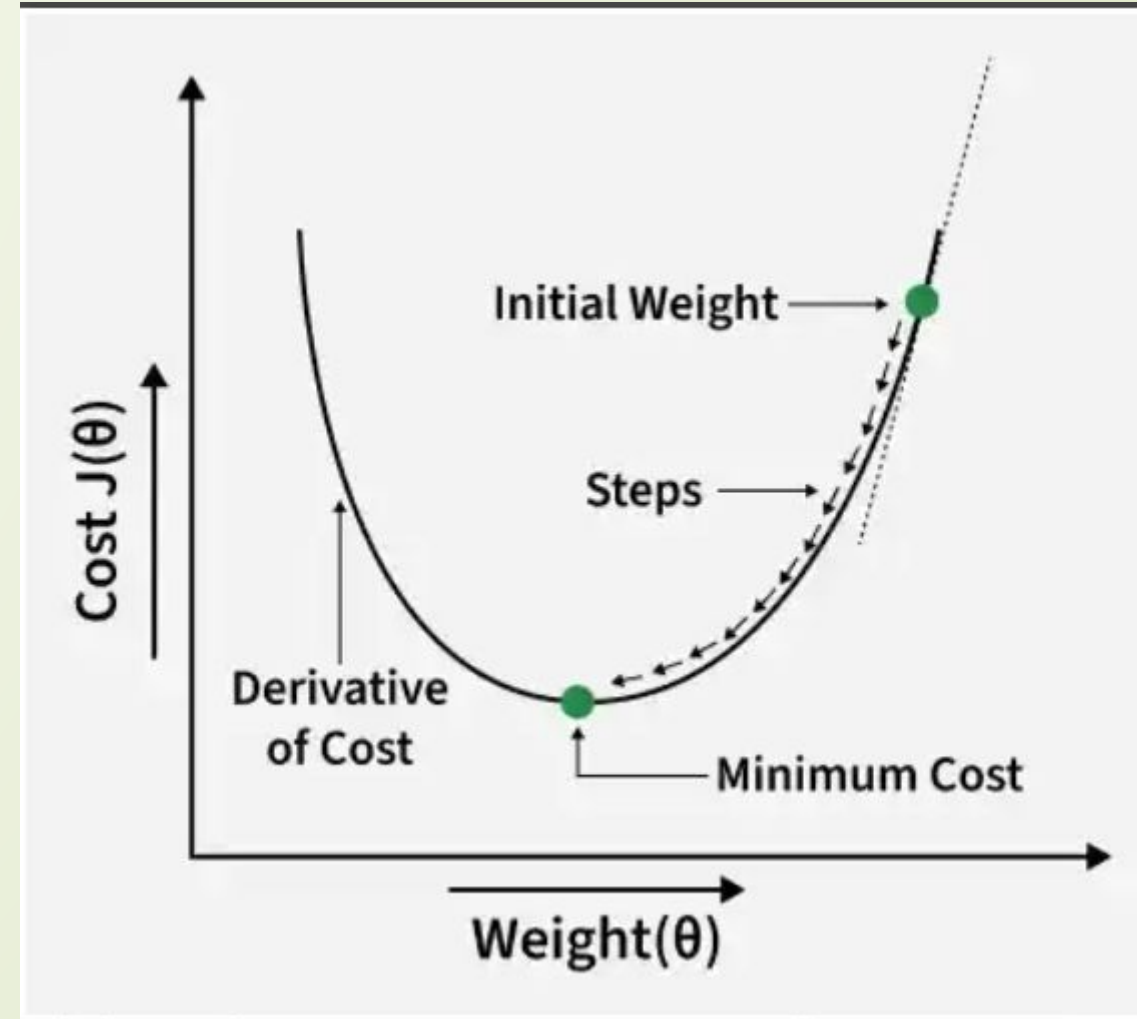

$$\text{Cost function}(J) = \frac{1}{n} \sum_n (\hat{y}_i - y_i)^2$$

Here:

- $\hat{y}_i = \theta_1 + \theta_2 x_i$: It is used to minimize this cost, we use Gradient Descent, which iteratively updates θ_1 and θ_2 until the MSE reaches its lowest value. This ensures the line fits the data as accurately as possible.

Gradient Descent

- Gradient descent is an optimization technique used to **train a linear regression model by minimizing the prediction error.**
- It works by starting with random model parameters and repeatedly adjusting them to reduce the difference between predicted and actual values.



How it works:

1. Start with random values for slope and intercept.
2. Calculate the error between predicted and actual values.
3. Find how much each parameter contributes to the error (gradient).
4. Update the parameters in the direction that reduces the error.
5. Repeat until the error is as small as possible.
6. This helps the model find the best-fit line for the data.

Evaluation Metrics

A variety of evaluation measures can be used to determine the strength of any linear regression model. These assessment metrics often give an indication of how well the model is producing the observed outputs.

1. **Mean Squared Error (MSE)**: Measures the average squared difference between actual and predicted values to avoid cancellation of errors.
2. **Mean Absolute Error (MAE)**: Calculate the accuracy of a regression model. MAE measures the average absolute difference between the predicted values and actual values.
3. **Root Mean Squared Error (RMSE)**: Square root of the residuals variance is RMSE. It describes how well the observed data points match the expected values or the model's absolute fit to the data.
4. **R-Squared**: Indicates how much variation the model explains. Its value is typically between 0 and 1, but it can be negative if the model performs worse than a simple baseline model (e.g., predicting the mean).
5. **Adjusted R-square**: Measures the proportion of variance explained by the model while adjusting for the number of predictors and penalizing irrelevant features.

Applications

- ❑ **Real Estate Price Prediction:** Estimate house prices using features such as location, area, and number of rooms.
- ❑ **Sales and Demand Forecasting:** Predict future sales and product demand based on historical trends and seasonal data.
- ❑ **Financial Analysis:** Analyze market trends, stock prices, and the impact of economic factors like inflation and interest rates.
- ❑ **Healthcare and Medical Research:** Predict disease progression, patient outcomes, and treatment effectiveness using medical data.
- ❑ **Marketing and Advertising Analysis:** Measure how advertising campaigns and promotional spending influence sales and customer engagement.

Confusion Matrix

A **confusion matrix** is used to evaluate the performance of a **classification model** by comparing the **actual values** with the **predicted values**.

Real-Time Example: Email Spam Detection

- ❑ Suppose an AI model classifies emails as **Spam** or **Not Spam**.
- ❑ You have **100 emails**.



Predicted Actual	Spam	Not Spam
Spam	35	5
Not Spam	10	50

Understanding Each Term

Term	Meaning	Example
True Positive (TP)	Model correctly predicts Spam	35 spam emails correctly identified as spam
True Negative (TN)	Model correctly predicts Not Spam	50 normal emails correctly identified
False Positive (FP)	Model predicts Spam but email is actually Not Spam	10 normal emails marked as spam
False Negative (FN)	Model predicts Not Spam but email is actually Spam	5 spam emails missed

Performance Metrics

1. Accuracy

How many predictions were correct?

$$\begin{aligned} \text{Accuracy} &= \frac{TP + TN}{TP + TN + FP + FN} \\ &= \frac{35 + 50}{35 + 50 + 10 + 5} = \frac{85}{100} = 85\% \end{aligned}$$

2. Precision

Out of all emails predicted as spam, how many were actually spam?

$$\begin{aligned} \text{Precision} &= \frac{TP}{TP + FP} \\ &= \frac{35}{35 + 10} = \frac{35}{45} = 77.78\% \end{aligned}$$

3. Recall (Sensitivity)

Out of all actual spam emails, how many were detected?

$$\begin{aligned} \text{Recall} &= \frac{TP}{TP + FN} \\ &= \frac{35}{35 + 5} = \frac{35}{40} = 87.5\% \end{aligned}$$

4. F1 Score

Balances Precision and Recall.

$$F1 = 2 \times \frac{Precision \times Recall}{Precision + Recall}$$

$$= 2 \times \frac{0.7778 \times 0.875}{0.7778 + 0.875} \approx 82.3\%$$

Real-Time Example: Disease Detection

- Suppose an AI predicts whether a patient has **COVID-19**.

Actual / Predicted	Positive	Negative
Positive	90	10
Negative	15	85

- 
1. **TP = 90:** Sick patients correctly identified.
 2. **TN = 85:** Healthy patients correctly identified.
 3. **FP = 15:** Healthy patients wrongly diagnosed as sick.
 4. **FN = 10:** Sick patients missed by the model.

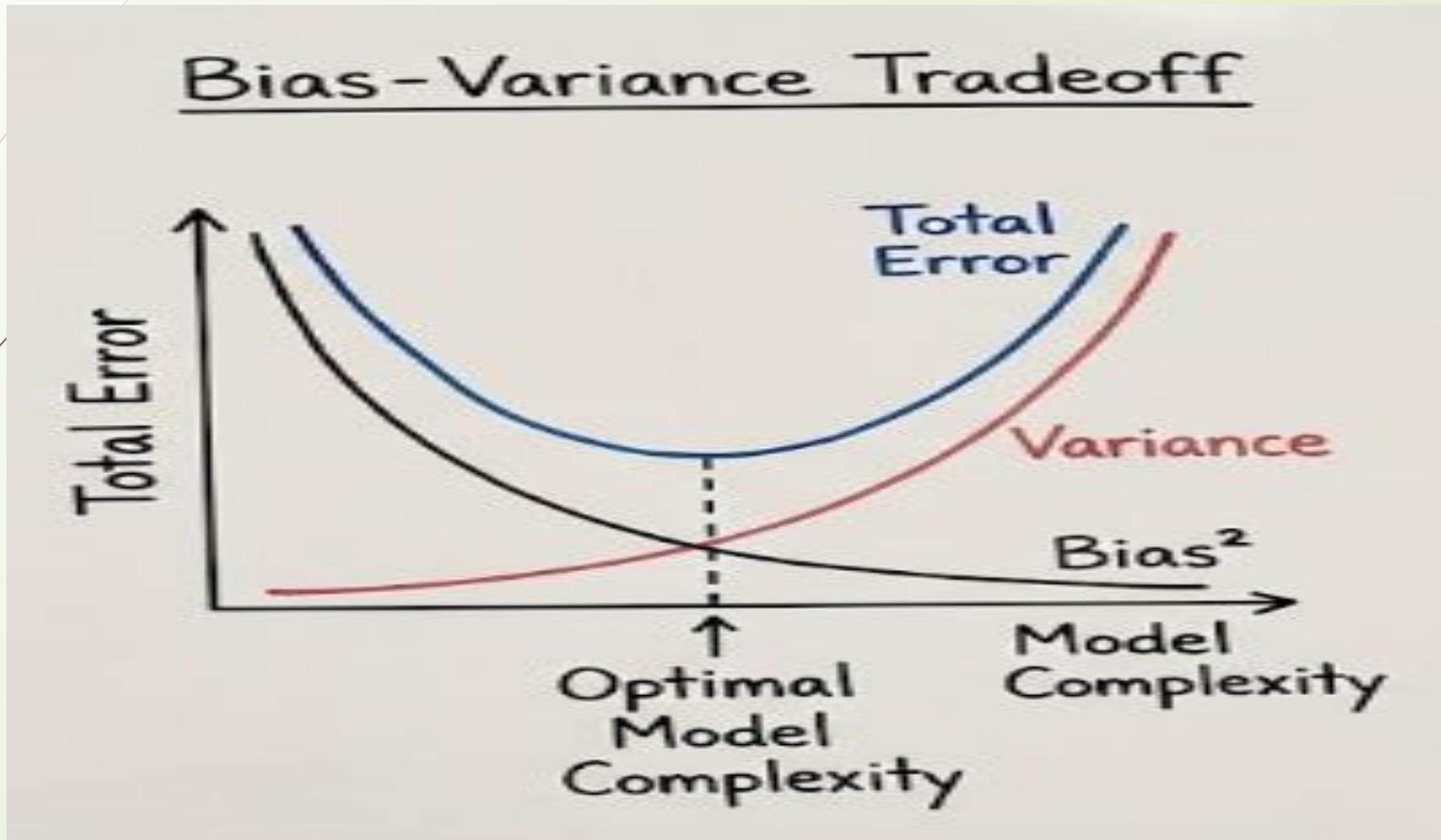
In healthcare, **False Negatives** are often the most dangerous because an infected patient may not receive treatment.

- 
- **TP (True Positive):** Correct positive prediction.
 - **TN (True Negative):** Correct negative prediction.
 - **FP (False Positive):** Incorrect positive prediction (False Alarm).
 - **FN (False Negative):** Incorrect negative prediction (Missed Detection).

Imagine a **COVID** test :

1. **TP**: Sick person → Test says **Sick** ✓
2. **TN**: Healthy person → Test says **Healthy** ✓
3. **FP**: Healthy person → Test says **Sick** ✗ (False Alarm)
4. **FN**: Sick person → Test says **Healthy** ✗ (Missed Case)

Bias – Variance Tradeoff



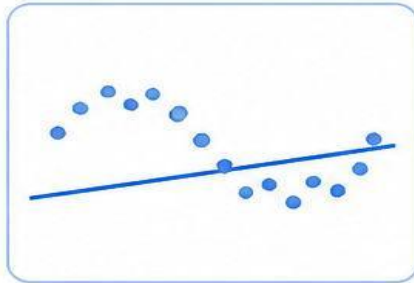
Bias-Variance Tradeoff

Balancing model complexity to **minimize total error**



Bias

Error from overly simple assumptions
(Underfitting)



High Bias

Model too simple
Misses patterns



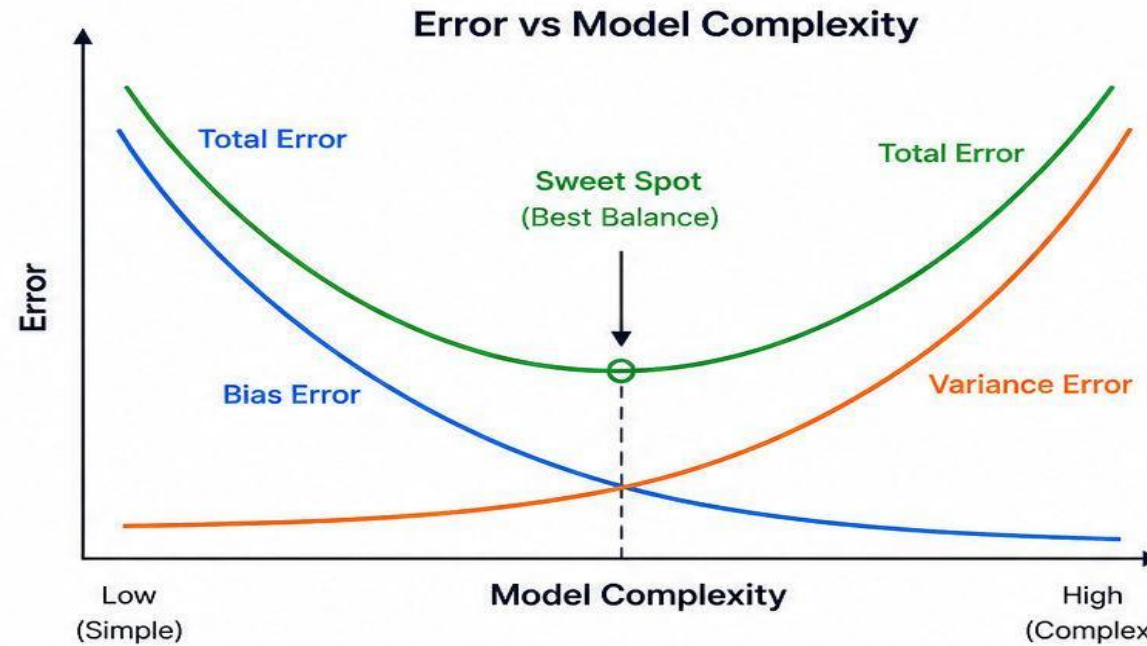
Variance

Error from sensitivity to training data
(Overfitting)



High Variance

Model too complex
Fits noise



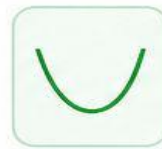
What Happens



Underfitting

High Bias
High Error

Model is too simple
Can't capture patterns



Just Right

Balanced Bias & Variance
Lowest Total Error

Model generalizes well
to new data



Overfitting

High Variance
High Error

Model is too complex
Fits noise, not signal

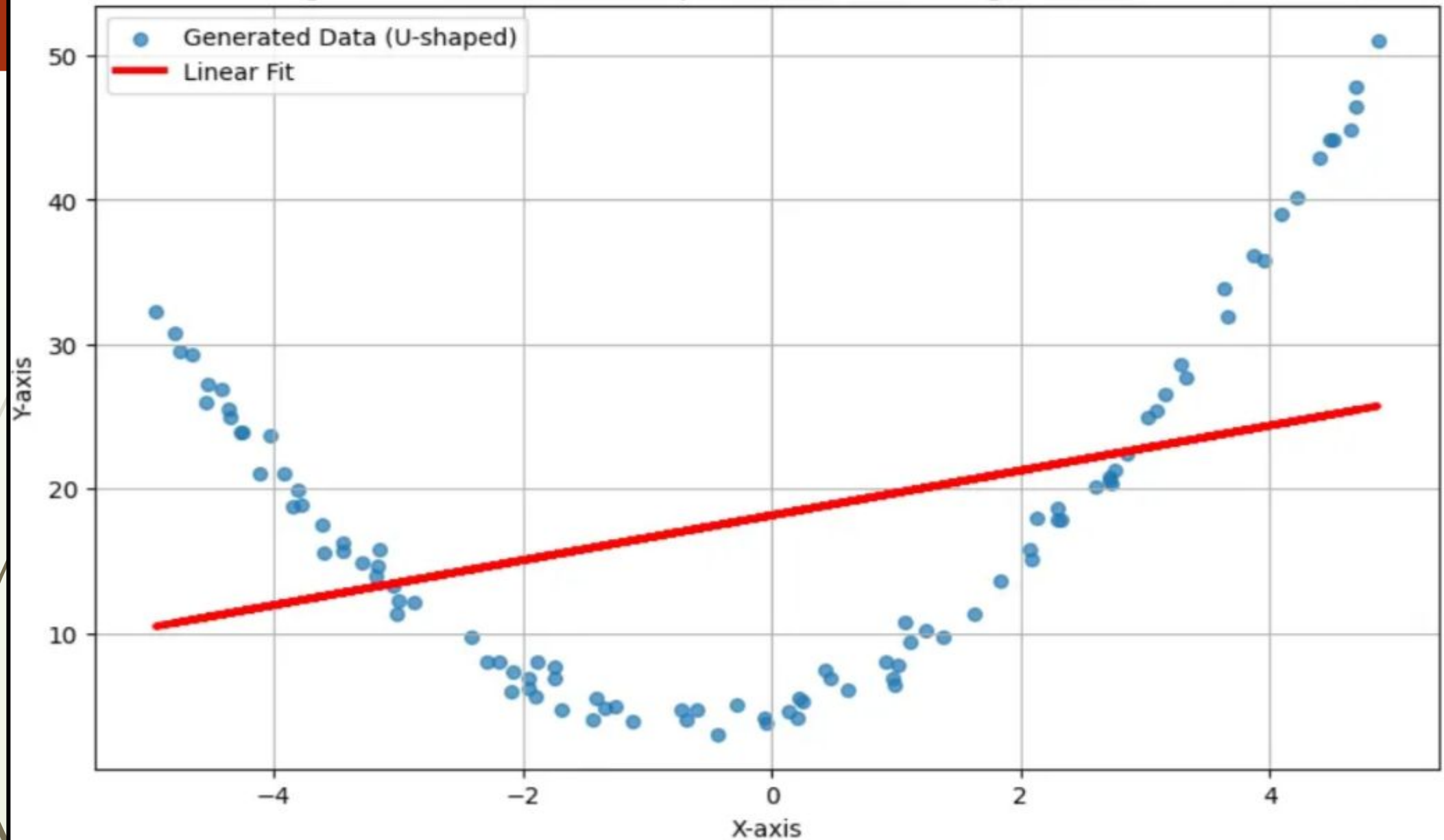
Bias and Variance .

- Before we dive into how to fix our models with regularization, we have to understand the two “ghosts” that haunt every machine learning algorithm: **Bias** and **Variance**.
- Every time we train a model, we are trying to minimize the **Total Error**. As your notes correctly show, this error is composed of three parts:
 - **Bias²** (Reducible)
 - **Variance** (Reducible)
 - **Irreducible Error** (Noise that we can't do anything about)

1. What is Bias? (The “Underfitter”)

- Bias is the error introduced by approximating a real-life problem (which is usually complicated) with a much simpler model.
- **The Problem:** The model makes too many assumptions. It’s “prejudiced” about what the data should look like.
- **Result: Underfitting.** No matter how much data you give it, **it just can’t learn the pattern.**
- **Visual:** Think of a straight line trying to fit a curved U-shape data set. It’s just too simple to get it right.

Illustrating Bias: Linear Fit on U-shaped Data (Underfitting - Curvier, Less Variance)

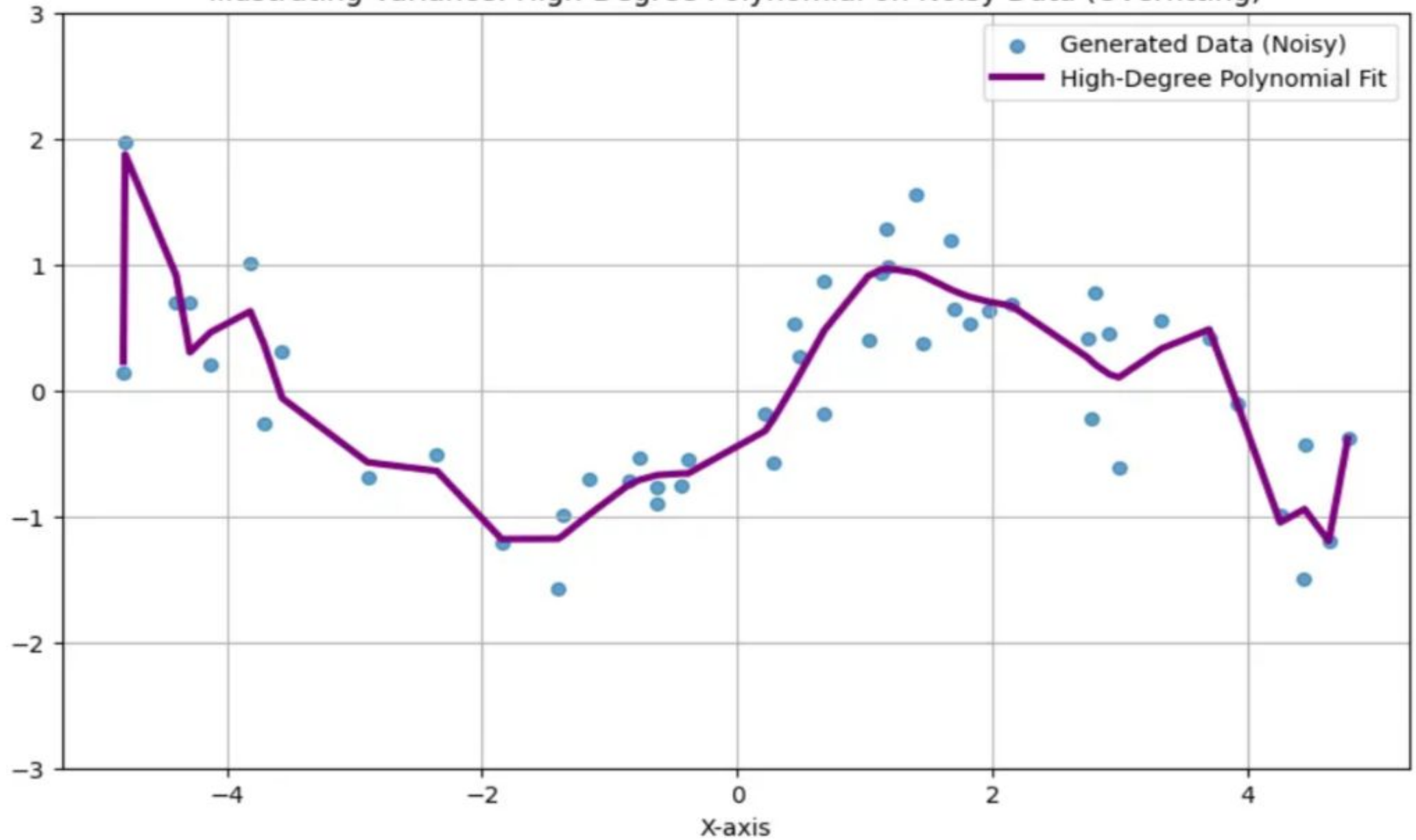


2. What is Variance? (The “Overfitter”)

Variance is the model’s sensitivity to small fluctuations in the training set.

- 1. The Problem:** The model learns the “noise” in the data rather than just the signal. It follows every single data point like a hyper-active puppy.
- 2. Result: Overfitting.** The model performs amazingly on training data but **fails miserably on new, unseen data.**
- 3. Visual:** A wiggly, complex line that passes through every single point on your graph but looks like a mess.

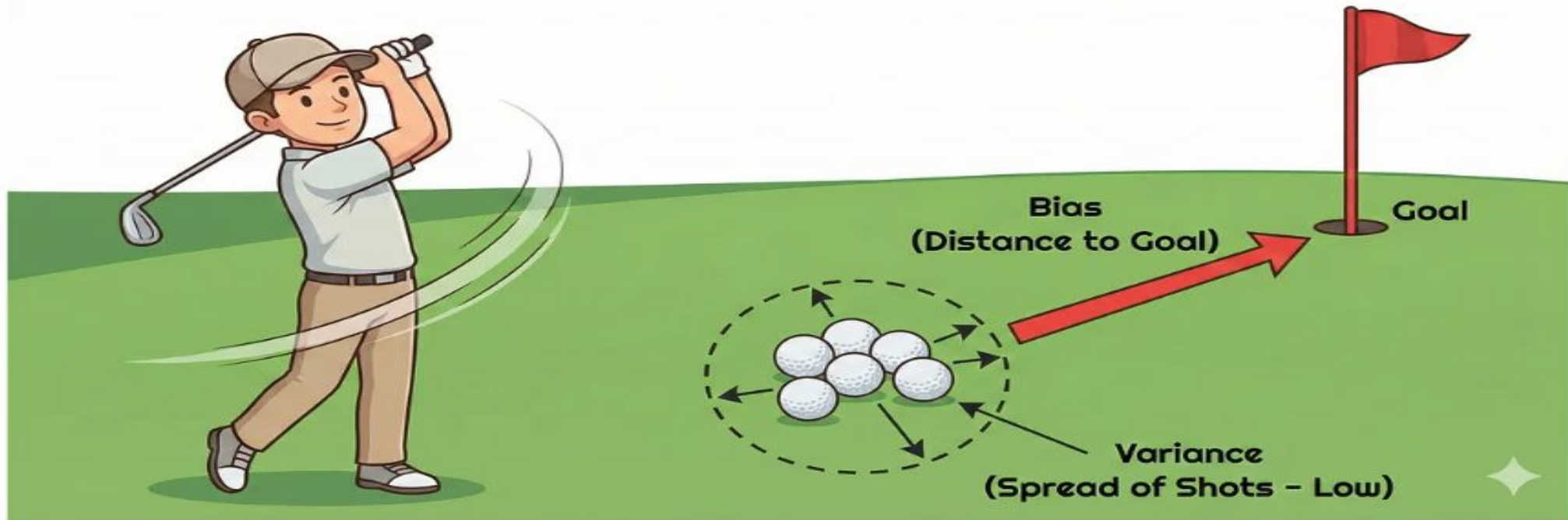
Illustrating Variance: High-Degree Polynomial on Noisy Data (Overfitting)



The Golf Pro Analogy

To visualize this, imagine a golfer taking several practice shots toward a hole. In this scenario, the hole represents the **true relationship** in your data.

Golf Example: High Bias, Low Variance

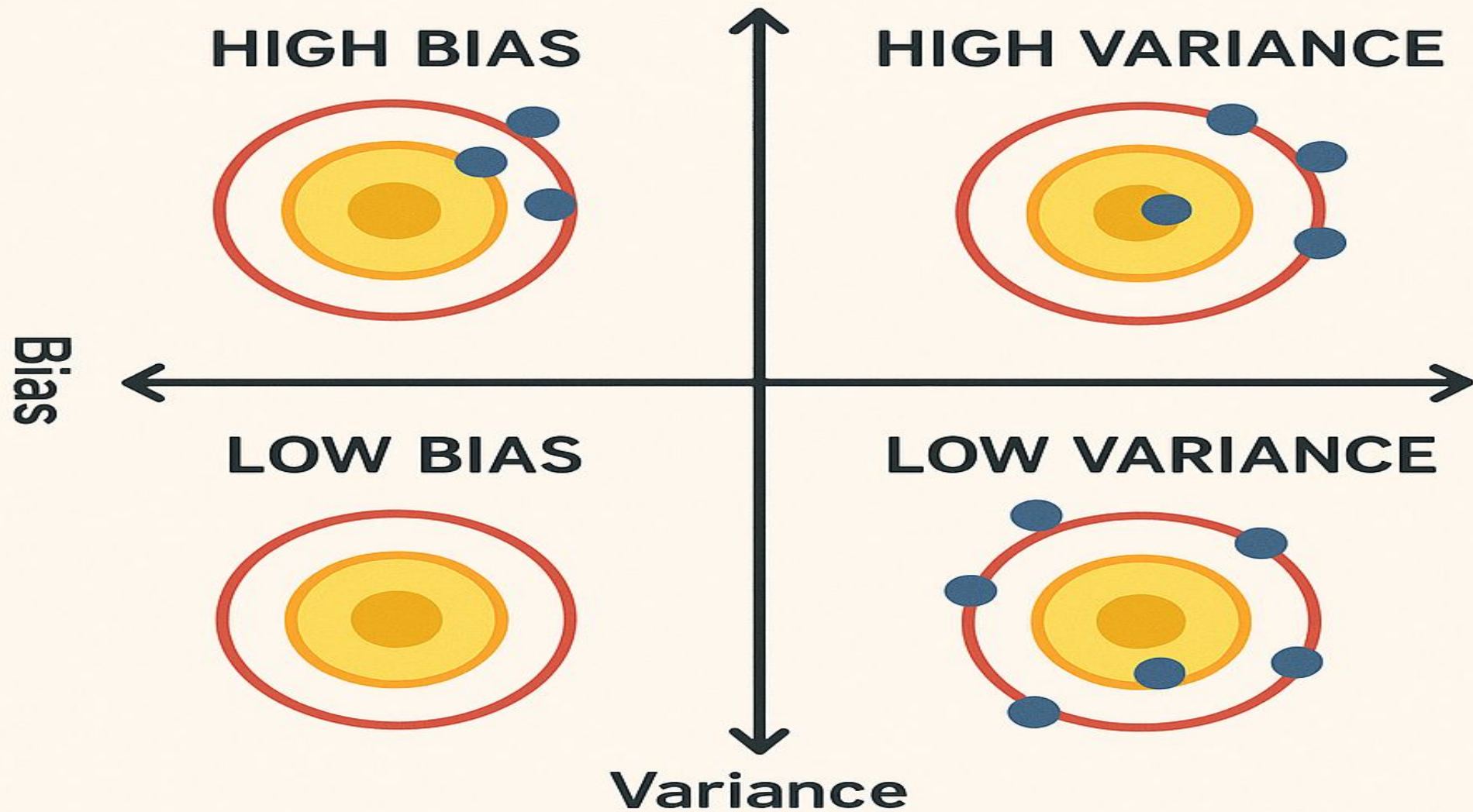


- **Bias** is the distance between the center of your ball cluster and the hole.
- **Variance** is the “spread” or how scattered the balls are from each other.

The Four Scenarios on the Green:

- 1. Low Bias / Low Variance (The Pro):** All the golf balls land exactly in or right next to the hole. The golfer is both accurate (centered on the goal) and consistent (no spread).
- 2. Low Bias / High Variance (The Wild Hitter):** The balls are scattered all over the green, but they are “centered” around the hole. On average, the aim is correct, but the individual shots are all over the place.
- 3. High Bias / Low Variance (The Consistent Miss):** All the balls land in a tiny, tight cluster... but they are 20 feet to the left of the hole. The golfer is very consistent, but systematically wrong. **This is Underfitting.**
- 4. High Bias / High Variance (The Amateur):** The balls are scattered everywhere, and they aren't even close to the hole. This is the worst-case scenario where the model has no idea what is going on.

THE BIAS-VARIANCE TRADEOFF EXPLAINED VISUALLY



Why do we need to “Balance” them?

- ❑ In a perfect world, we want **Low Bias and Low Variance**. However, in reality, there is a tug-of-war:
- ❑ As you make a model more **complex** (to reduce Bias), it starts to pick up noise, and **Variance increases**.
- ❑ As you make a model **simpler** (to reduce Variance), it loses its ability to learn, and **Bias increases**.
- ❑ *The Sweet Spot: We want to find the point where the sum of Bias² and Variance is at its lowest. This is the “Best Model Complexity” point.*

The Bridge to Regularization

- When we find ourselves in a situation with **High Variance (Overfitting)**, our model is being **too “flexible.”** It’s trying too hard to please **every data point in the training set.**
- This is exactly where **Regularization (L1 and L2)** comes in. Regularization acts like a “penalty” or a “leash” that prevents the model from becoming too complex. It forces the model to stay simpler, effectively trading a tiny bit of Bias to significantly reduce the Variance.

The Problem: High Variance & Overfitting

- When a model is too complex (like a high-degree polynomial), it begins to “memorize” **the noise and outliers in the training set** rather than learning the underlying pattern. Mathematically, this manifests as exceptionally large coefficients (W).
- Regularization solves this by adding a **penalty term** to our Loss Function (L). Instead of just minimizing the error, we now minimize:

$$L = \text{Residual Sum of Squares (RSS)} + \text{Penalty}$$

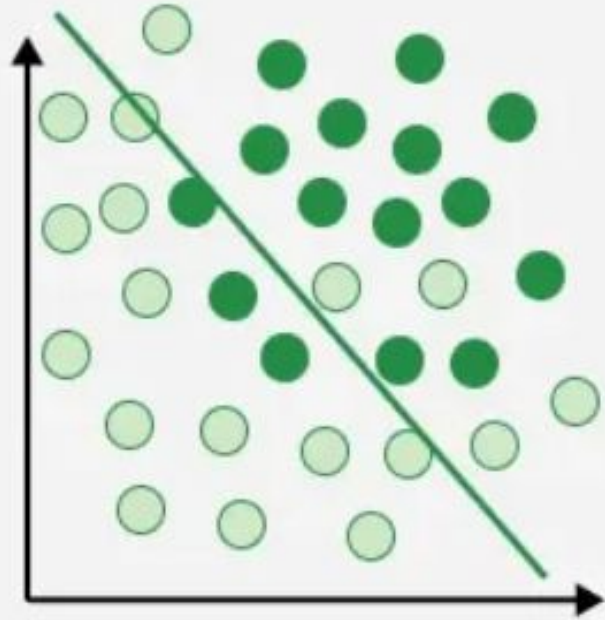
Regularization

- ❑ Regularization is a technique used in machine **learning to prevent overfitting**, which otherwise causes models to perform poorly on unseen data.
- ❑ By adding a penalty for complexity, regularization encourages simpler and more generalizable models.
- ❑ **Prevents overfitting:** Adds constraints to the model to reduce the risk of memorizing noise in the training data.
- ❑ **Improves generalization:** Encourages simpler models that perform better on new, unseen data.

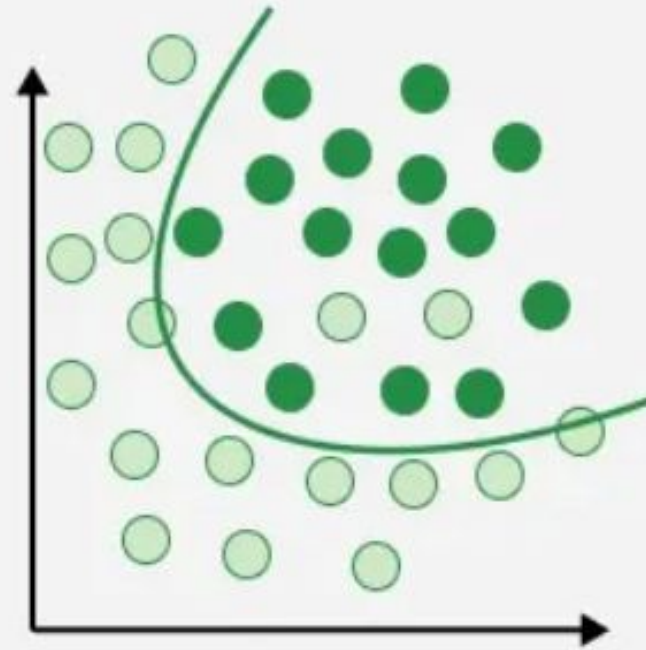
Regularization Techniques

- ❑ **Lasso Regression**: Regularizes a linear regression model, it adds a penalty term to the linear regression objective function to prevent overfitting.
- ❑ **Ridge regression**: Adds a regularization term to the standard linear objective to prevent overfitting by penalizing large coefficient in linear regression equation. It useful when the dataset has multicollinearity where predictor variables are highly correlated.
- ❑ **Elastic Net Regression**: Hybrid regularization technique that combines the power of both L1 and L2 regularization in linear regression objective

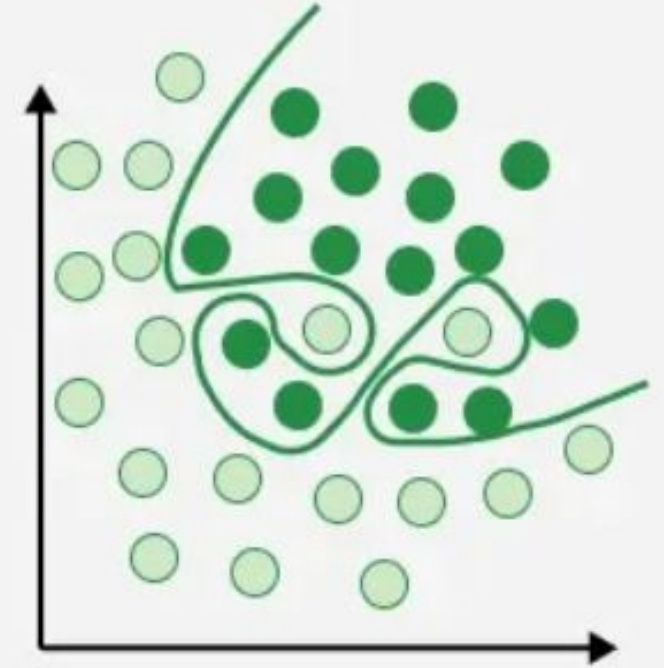
Regularization in Machine Learning



Under-fitting



Appropriate-fitting



Over-fitting

Types of Regularization

- There are mainly 3 types of regularization techniques, each applying penalties in different ways to control model complexity and improve generalization.

1. Lasso Regression

A regression model which uses the L1 Regularization technique is called **LASSO (Least Absolute Shrinkage and Selection Operator)** regression.

1. It adds the absolute value of magnitude of the coefficient as a penalty term to the loss function(L).
2. This penalty can shrink some coefficients to zero which helps in selecting only the important features and ignoring the less important ones.

Lasso Regression

$$\text{Cost} = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2 + \lambda \sum_{j=1}^m |w_j|$$

Where

- m : Number of Features
- n : Number of Examples
- y_i : Actual Target Value
- $\hat{y}_i$: Predicted Target Value



```
from sklearn.linear_model import Lasso
from sklearn.model_selection import train_test_split
from sklearn.datasets import make_regression
from sklearn.metrics import mean_squared_error

X, y = make_regression(n_samples=100, n_features=5, noise=0.1, random_state=42)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

lasso = Lasso(alpha=0.1)
lasso.fit(X_train, y_train)

y_pred = lasso.predict(X_test)

mse = mean_squared_error(y_test, y_pred)
print(f"Mean Squared Error: {mse}")

print("Coefficients:", lasso.coef_)
```

Mean Squared Error: 0.06362439921332456

Coefficients: [60.50305581 98.52475354 64.3929265 56.96061238 35.52928502]

- 
- ❑ **X, y = make_regression(n_samples=100, n_features=5, noise=0.1, random_state=42):** Generates a regression dataset with 100 samples, 5 features and some noise.
 - ❑ **X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42):** Splits the data into 80% training and 20% testing sets.
 - ❑ **lasso = Lasso(alpha=0.1):** Creates a Lasso regression model with regularization strength alpha set to 0.1.
 - ❑ The output shows the model's prediction error and the importance of features with some coefficients reduced to zero due to L1 regularization

2. Ridge Regression

- A regression model that uses the L2 regularization technique is called Ridge regression.
- It adds the squared magnitude of the coefficient as a penalty term to the loss function(L).
- It handles multicollinearity by shrinking the coefficients of correlated features, reducing their variance and preventing any single feature from dominating the model.

$$\text{Cost} = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2 + \lambda \sum_{j=1}^m w_j^2$$


$$\text{Cost} = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2 + \lambda \sum_{j=1}^m w_j^2$$

Where,

n: Number of examples or data points

m: Number of features i.e predictor variables

y_i : Actual target value for the i th example

$\hat{y}_i$: Predicted target value for the i th example

w_i : Coefficients of the features

λ : Regularization parameter that controls the strength of regularization

□ The output shows the MSE showing model performance. **Lower MSE means better accuracy.** The coefficients reflect the regularized feature weights.

- `ridge = Ridge(alpha=1.0)`: Creates a Ridge regression model with regularization strength α set 1.0.

```
from sklearn.linear_model import Ridge
from sklearn.datasets import make_regression
from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_squared_error

X, y = make_regression(n_samples=100, n_features=5, noise=0.1, random_state=42)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

ridge = Ridge(alpha=1.0)
ridge.fit(X_train, y_train)
y_pred = ridge.predict(X_test)

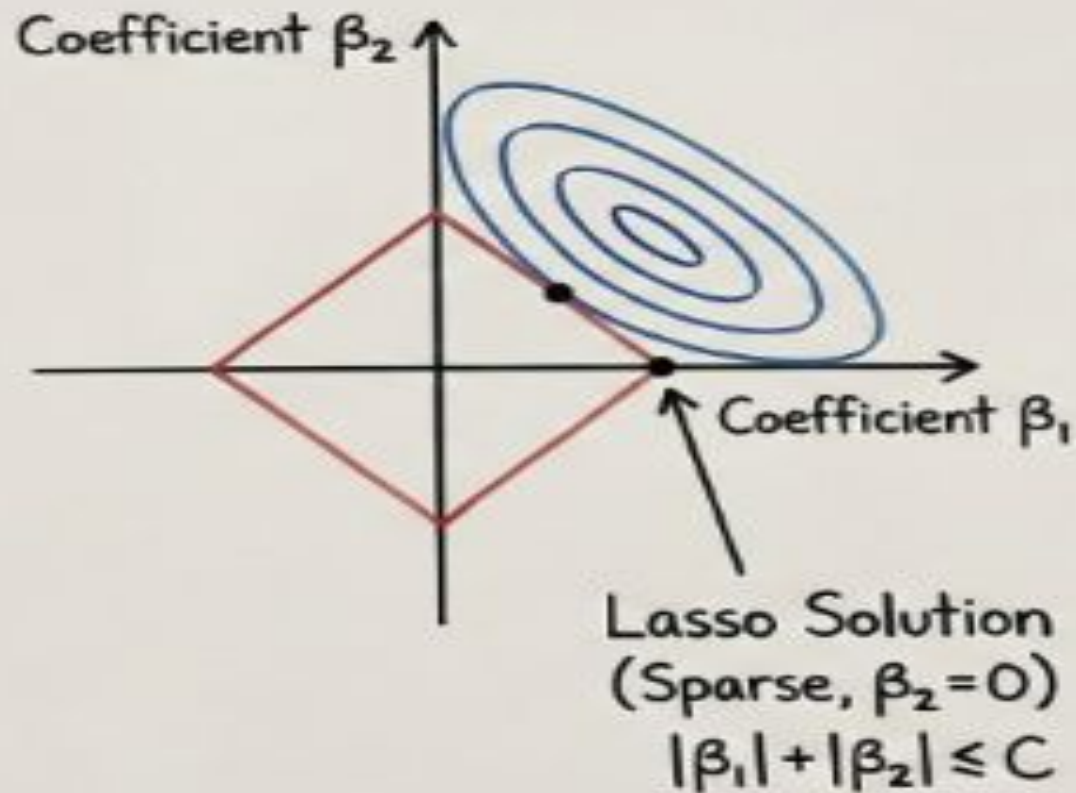
mse = mean_squared_error(y_test, y_pred)
print("Mean Squared Error:", mse)
print("Coefficients:", ridge.coef_)
```

Output:

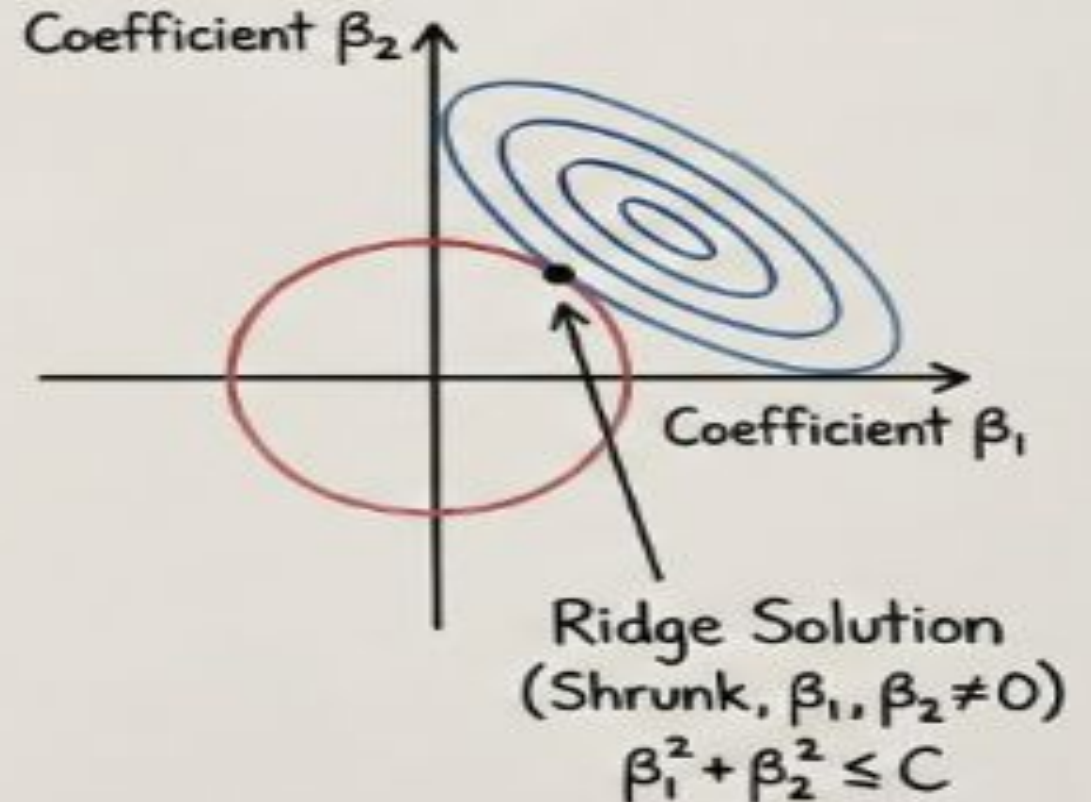
```
Mean Squared Error: 4.114050771972589
Coefficients: [59.87954432  97.15091098  63.24364738  56.31999433  35.34591136]
```

Lasso and Ridge

L1 Regularization (Lasso)
- Feature Selection



L2 Regularization (Ridge)
- Coefficient Shrinkage



3. Elastic Net Regression

- Elastic Net Regression is a combination of both L1 as well as L2 regularization.
- It combines both L1 (**absolute values**) and L2 (**squared values**) penalties on the coefficients. With the help of an extra hyperparameter that controls the ratio of the L1 and L2 regularization.

$$\text{Cost} = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2 + \lambda \left((1 - \alpha) \sum_{j=1}^m |w_j| + \alpha \sum_{j=1}^m w_j^2 \right)$$

Where

- n : Number of examples (data points)
- m : Number of features (predictor variables)
- y_i : Actual target value for the i^{th} example
- $\hat{y}_i$: Predicted target value for the i^{th} example
- w_i : Coefficients of the features
- λ : Regularization parameter that controls the strength of regularization
- α : Mixing parameter where $0 \leq \alpha \leq 1$ and $\alpha = 1$ corresponds to Lasso (L_1) regularization, $\alpha = 0$ corresponds to Ridge (L_2) regularization and Values between 0 and 1 provide a balance of both L1 and L2 regularization

- **model = ElasticNet(alpha=1.0, l1_ratio=0.5)** : Creates an Elastic Net model with regularization strength $\alpha=1.0$ and L1/L2 mixing ratio 0.5.

```
from sklearn.linear_model import ElasticNet
from sklearn.datasets import make_regression
from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_squared_error

X, y = make_regression(n_samples=100, n_features=10, noise=0.1, random_state=42)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

model = ElasticNet(alpha=1.0, l1_ratio=0.5)
model.fit(X_train, y_train)

y_pred = model.predict(X_test)
mse = mean_squared_error(y_test, y_pred)

print("Mean Squared Error:", mse)
print("Coefficients:", model.coef_)
```

Output:

```
Mean Squared Error: 7785.886176938014
Coefficients: [16.84528938 31.77080959  4.05901996 40.18486737 57.25856154 45.81463318
 58.97979422 -0.          3.82816854 41.1096051 ]
```


Benefits of Regularization

- 1. Prevents Overfitting:** Regularization helps models focus on underlying patterns instead of memorizing noise in the training data.
- 2. Enhances Performance:** Prevents excessive weighting of outliers or irrelevant features helps in improving overall model accuracy.
- 3. Stabilizes Models:** Reduces sensitivity to minor data changes which ensures consistency across different data subsets.
- 4. Prevents Complexity:** Keeps model from becoming too complex which is important for limited or noisy data.
- 5. Handles Multicollinearity:** Reduces the magnitudes of correlated coefficients helps in improving model stability.
- 6. Promotes Consistency:** Ensures reliable performance across different datasets which reduces the risk of large performance shifts.

